CVR COLLEGE OF ENGINEERING

(UGC Autonomous Institution, Accredited by NAAC 'A' grade and NBA)
Affiliated to JNTU Hyderabad
Vastunagar, Mangalpalli (V), Ibrahimpatnam (M),
Ranga Reddy (Dist.), Hyderabad – 501510, Telangana State.

IV B.Tech II Semester

Major Project Work

Department of Computer Science and Engineering

2025-2026

User-Aware Topic Modeling on Transliteration-Based Telugu Text using BERTopic

A dissertation submitted to the Jawaharlal Nehru Technological University, Hyderabad in partial fulfillment of the requirement for the award of degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted
by

Allam Abhinay(22B81A05R9)

Esha Thallapureddy (22B81A05T0)

Bangaru Gunasai Sudheerthi (22B81A05T2)

Under the Guidance of
I B N Hima Bindu
Sr. Assistant Professor

Department of Computer Science and Engineering

CVR COLLEGE OF ENGINEERING

(An UGC Autonomous Institution, Affiliated to JNTUH, Accredited by NBA, and NAAC)
Vastunagar, Mangalpalli (V), Ibrahimpatnam (M),
Ranga Reddy (Dist.) - 501510, Telangana State

2025-2026

CVR COLLEGE OF ENGINEERING

(An UGC Autonomous Institution, Affiliated to JNTUH, Accredited by NBA, and NAAC)
Vastunagar, Mangalpalli (V), Ibrahimpatnam (M),
Ranga Reddy (Dist.) - 501510, Telangana State.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project Work entitled **User-Aware Topic Modeling on Transliteration-Based Telugu Text using BERTopic** is submitted by **Allam Abhinay (22B81A05R9), Esha Thallapureddy (22B81A05T0), and Bangaru Gunasai Sudheerthi (22B81A05T2)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** for the academic year 2025–2026

Project Supervisor
I B N Hima Bindu
Sr. Assistant Professor

Professor in-charge project
Dr. M. Sridevi
Professor

Professor and Head, CSE
(Dr. A. Vani Vathsala)

DECLARATION

We, hereby declare that this Project Work report titled **User-Aware Topic Modeling on Transliteration-Based Telugu Text using BERTopic** submitted to the Department of Computer Science and Engineering, CVR College of Engineering, is a record of original work done by me under the guidance of **I B N Hima Bindu.** The information and data given in the report is authentic to the best of my knowledge. This project report has not been submitted to any other university or institution for the award of any degree or diploma, nor has it been published previously.

Allam Abhinay (22B81A05R9)

Esha Thallapureddy (22B81A05T0)

Bangaru Gunasai Sudheerthi (22B81A05T2)

Date:

Place:

ACKNOWLEDGEMENT

We sincerely thank **Dr. Ramamohan Reddy Kasa, Principal,** CVR College of Engineering, for his cooperation and encouragement throughout the project.

We earnestly thank **Dr. A Vani Vathsala, HOD,** Department of CSE, CVR College of Engineering, for giving timely cooperation and taking necessary action throughout the course of our project.

We express our sincere thanks and gratitude to our Mini-Project Coordinator **Ms. G. Deepika, Assistant Professor** and Supervisor **I B N Hima Bindu, Sr. Assistant Professor,** Department of CSE, CVR College of Engineering, for valuable help and encouragement throughout the project work.

Finally, we place in records our sincere appreciation and indebtedness to our parents and all the Faculty Members for their understanding and cooperation, without whose encouragement and blessing it would have been impossible to complete this work.

With Regards,

22B81A05R9

22B81A05T0

22B81A05T2

ABSTRACT

Social media platforms today are flooded with content written in informal, mixed-language styles, especially among young users from multilingual communities. Telugu speakers in particular tend to write using a combination of Telugu words transliterated into English (Roman script) and actual English words, creating what is commonly called Tanglish. This kind of text is very different from the clean, formal text that most NLP tools are designed to handle. Standard topic modeling systems fail badly on Tanglish because they cannot make sense of the inconsistent spellings and mixed vocabulary.

This project, which we have called UATM (User-Aware Topic Modeling), tries to solve this problem. We built a complete system from scratch that can take raw Tanglish social media posts, clean and normalize them, identify what topics they are talking about, and then track how individual users' interests change over time. The system is built as a full web application with a FastAPI backend and a React frontend, and it includes a dashboard where both regular users and administrators can explore topic trends.

The core of the system is a three-stage normalization pipeline. In the first stage, we handle basic spelling corrections and remove noise. In the second stage, we group together words that sound alike but are spelled differently, which is very common in Tanglish. In the third stage, we use a transformer model to resolve ambiguous words using context. After normalization, we use BERTopic to extract topics. BERTopic works by converting text into vector embeddings using a pre-trained multilingual model, then reducing those high-dimensional embeddings using UMAP, and finally clustering them using HDBSCAN.

What makes our system unique is the user-aware component. After topic modeling, we compute a topic distribution for each user and track how it changes across different time windows. We use two measures for this: Shannon Entropy, which tells us how spread out a user's topics are, and Jensen-Shannon Divergence, which measures how much the distribution has shifted between two time periods. Users who cross a defined JSD threshold are flagged as having significant topic drift.

We tested the system thoroughly.The normalization pipeline improved topic coherence scores significantly, and the drift detection module was able to identify users with shifting interests accurately. This project is, to our knowledge, one of the first end-to-end systems specifically built for user-aware topic modeling on Tanglish social media text.

TABLE OF CONTENTS

List of Tables

List of Figures

ABBREVIATIONS AND SYMBOLS

Abbreviation	Full Form
UATM	User-Aware Topic Modeling
NLP	Natural Language Processing
BERTopic	Bidirectional Encoder Representations from Transformers Topic Modeling
BERT	Bidirectional Encoder Representations from Transformers
JSD	Jensen-Shannon Divergence
API	Application Programming Interface
REST	Representational State Transfer
FastAPI	Fast Application Programming Interface (Python Web Framework)
UI	User Interface
ML	Machine Learning
UMAP	Uniform Manifold Approximation and Projection
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise
TF-IDF	Term Frequency-Inverse Document Frequency
c-TF-IDF	Class-based Term Frequency-Inverse Document Frequency
JSON	JavaScript Object Notation
JWT	JSON Web Token
RBAC	Role-Based Access Control
LDA	Latent Dirichlet Allocation
RAM	Random Access Memory
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
CORS	Cross-Origin Resource Sharing
ORM	Object Relational Mapper

CHAPTER 1

INTRODUCTION

With the rapid growth of social media and online communication platforms, a massive amount of user-generated textual content is produced every day. This content contains valuable signals about user interests, opinions, community trends, and behavioral patterns. However, extracting meaningful information from this data is significantly challenging, especially when the text is written in code-mixed and transliterated forms, as is the norm among Telugu-speaking communities across Andhra Pradesh and Telangana.

Telugu speakers on social media rarely write in pure Telugu or pure English. Instead, they mix the two languages within the same sentence, and they write Telugu words using English letters based on pronunciation rather than any standardized orthography. This hybrid form is commonly called Tanglish, and it appears across platforms such as Twitter, YouTube, Facebook, and WhatsApp in massive volumes every day. From a Natural Language Processing perspective, Tanglish is extremely challenging. The same Telugu word might be spelled in a dozen different ways by different users, and the absence of any canonical script makes normalization and semantic analysis far more difficult than for standard languages.

Beyond the language challenge, there is a deeper analytical limitation in how topic modeling has traditionally been applied. Most existing topic modeling systems extract a single global set of topics from a corpus and assign documents to those topics uniformly. They do not take into account who produced the content, how a specific user's posting patterns differ from others, or how any individual user's interests shift over time. In practice, this user-level perspective is often more valuable than a global view. Understanding that a large number of users within a community have recently shifted their discussions from entertainment to politics, for instance, is a far richer insight than simply knowing that both topics exist in the corpus.

This project, titled "User-Aware Topic Modeling on Transliteration-Based Telugu Text using BERTopic", addresses both of these challenges simultaneously. We designed and implemented a system called UATM that combines a domain-specific Tanglish normalization pipeline with transformer-based topic modeling and user-aware temporal drift detection. The system is delivered as a full-stack web application with role-based dashboards that make the analytical outputs accessible to both technical administrators and non-technical end users.

1.1 Motivation

1.1.1 The Limitations of Global Topic Modeling

The primary motivation behind this project is the fundamental limitation of existing topic modeling approaches when applied to user-generated social media content. Traditional topic modeling techniques such as Latent Dirichlet Allocation treat all documents in a corpus symmetrically and produce a single global topic distribution. While this is appropriate for analyzing static document collections such as news archives or academic papers, it is poorly suited for social media platforms where the same corpus represents the output of thousands of distinct individuals with different interests, communication styles, and behavioral trajectories.

In practical social media analytics scenarios, the user dimension is often the most important dimension of analysis. A platform administrator who wants to understand why engagement on a particular topic is declining needs to know not just that the topic exists, but which users were discussing it, how their engagement evolved over time, and whether they have shifted their attention to other topics. None of this information is captured by a conventional topic model, which is the gap this project directly addresses.

1.1.2 The Challenge of Tanglish Text

A second and equally important motivation is the specific challenge posed by Tanglish text. Telugu is a Dravidian language spoken by approximately 85 million people, making it one of the most widely spoken languages in India. However, despite its large speaker base, Telugu remains significantly underrepresented in NLP research, particularly in the context of social media and code-mixed language processing. Most available NLP tools and pre-trained models are designed for high-resource languages such as English, Mandarin, and Spanish, and fail to handle the irregular, mixed-script nature of Tanglish.

The core difficulty is orthographic inconsistency. When Telugu speakers write in Roman script, there is no standardized mapping from Telugu phonemes to English letters. A single Telugu word might be written as five or ten different sequences of English characters by five or ten different users, all of which sound identical when spoken. This means that standard NLP preprocessing steps such as tokenization, stopword removal, and vocabulary-based lookup produce very poor results on Tanglish text. Building a reliable normalization pipeline specifically for Tanglish was therefore a necessary prerequisite for any downstream NLP task, including topic modeling.

1.1.3 The Need for a User-Aware, Deployable System

A third motivation was the lack of any deployable end-to-end system for topic modeling of Telugu social media content. Prior academic work in this space has produced isolated models, datasets, or scripts, but has not delivered a complete, integrated application that a researcher or analyst could actually use. The need to bridge this gap between research prototype and deployable system motivated our decision to build UATM as a full-stack web application with a React frontend and FastAPI backend, rather than as a standalone NLP pipeline. The goal was to create a system that is not only technically sound but practically usable.

1.2 Problem Statement

The construction and analysis of topic models from social media data in the Tanglish domain present a challenging problem with two distinct but interrelated dimensions.

The first dimension is the preprocessing challenge. Tanglish social media text is characterized by extreme orthographic variability, informal and unstructured sentence construction, mixing of Telugu and English vocabulary within the same sentence, liberal use of abbreviations and slang, and absence of any consistent transliteration convention. Traditional NLP pipelines designed for standard languages are unable to handle these characteristics. Standard tokenizers produce fragmented or incorrect tokens. Dictionary-based spell checkers fail because they have no knowledge of Tanglish vocabulary. Pre-trained word embeddings trained on English corpora produce poor or meaningless representations for Tanglish tokens. As a result, topic models trained directly on raw Tanglish text produce noisy, incoherent topics with low semantic validity.

The second dimension is the user-awareness challenge. Even when reasonably good topic models can be extracted from a corpus, existing systems do not incorporate the user dimension. They assign topics to documents without tracking which user produced each document, and they produce a single static topic distribution for the entire corpus rather than maintaining per-user distributions that evolve over time. This means that important behavioral signals, such as a user who has suddenly shifted their interests from cricket to politics, or a community where discussions of a particular topic have declined sharply over the past month, remain invisible.

The problem addressed by this project can therefore be defined as follows: how can we design an integrated system that accurately normalizes Tanglish text to enable reliable topic modeling, and simultaneously tracks per-user topic distributions over time to detect and surface meaningful shifts in user interests? The system must be accurate enough to produce semantically coherent topics, sensitive enough to detect genuine behavioral drift, and deployable enough to be used by non-technical analysts through a web-based interface.

1.3 Project Objectives

The primary objective of this project is to design and implement UATM, a user-aware topic modeling system for Tanglish social media text. The specific objectives are organized as follows:

1.3.1 Develop a Three-Tier Tanglish Normalization Pipeline:

Design and implement a sequential normalization pipeline with three distinct tiers: lexical normalization using a custom dictionary, phonetic normalization using Soundex and Metaphone encoding, and contextual normalization using a multilingual masked language model. The pipeline must reduce lexical diversity in the input corpus and improve the semantic coherence of downstream topic models.

1.3.2 Integrate BERTopic for Semantic Topic Modeling:

Integrate the BERTopic framework with a multilingual sentence transformer embedding model to perform topic modeling on the normalized Tanglish text. The integration must be configured to handle the specific characteristics of the Tanglish corpus, including appropriate embedding model selection, UMAP dimensionality reduction parameters, and HDBSCAN clustering configuration. The system must achieve a C_v topic coherence score that substantially improves on both LDA-based baselines and BERTopic without normalization.

1.3.3 Implement User-Aware Topic Distribution Tracking:

After topic assignment, maintain per-user topic probability distributions computed over sliding time windows. For each user, compute Shannon Entropy over their current topic distribution to measure interest diversity, and compute Jensen-Shannon Divergence between consecutive distributions to measure interest shift. Generate drift alerts for users whose JSD exceeds a configurable threshold.

1.3.4 Build a Full-Stack Web Application with Role-Based Dashboards:

Develop a deployable full-stack application using FastAPI as the backend framework and React as the frontend framework. The application must implement JWT-based authentication with role-based access control, providing separate dashboards for administrators and regular users. The dashboards must present topic distributions, entropy trends, and drift alerts in a visually intuitive format accessible to non-technical users.

1.3.5 Evaluate and Validate the System Comprehensively:

Evaluate the normalization pipeline against lexical diversity reduction metrics, evaluate the topic modeling component against C_v coherence scores with comparisons to LDA and unnormalized BERTopic baselines, and evaluate the drift detection module against precision and recall on a labeled longitudinal evaluation dataset. Validate the full system against all defined functional requirements through unit testing, integration testing, and system-level testing.

1.4 Project Report Organization

This report is organized in a structured manner to provide a comprehensive understanding of the UATM system, from its conceptual foundations through its design, implementation, and evaluation. The report follows a logical progression designed to guide the reader from the problem context to the proposed solution and its validated outcomes.

The report begins with this Introduction, which establishes the context, motivation, problem statement, and objectives of the project. It explains the need for integrating Tanglish-specific normalization with transformer-based topic modeling and user-aware drift detection, and highlights the limitations of existing approaches in addressing these combined challenges.

Chapter 2, Literature Survey, presents a systematic review of prior work across four research areas: traditional topic modeling approaches including LDA, BTM, and NMF; transformer-based topic modeling methods including BERTopic, Top2Vec, and CTM; code-mixed and transliterated NLP research covering language identification, normalization, and Telugu-specific work; and user modeling and temporal drift detection. Each area is analyzed to identify specific limitations that motivated design decisions in UATM.

Chapter 3, Requirement Analysis, documents the requirements identified for the UATM system. This includes a table of fourteen functional requirements with unique identifiers, five non-functional requirements organized into subsections covering performance, usability, security, reliability, and maintainability, detailed software and hardware requirement tables, and specific requirements for both the Administrator and Regular User roles.

Chapter 4, System Design, describes the proposed architecture and the design of each major system component. It covers the three-layer client-server architecture and data flow, the three-tier normalization pipeline in detail, the BERTopic integration including embedding model selection and clustering configuration, the mathematical design of the drift detection module including the JSD and entropy formulations, UML diagrams for use case, activity, sequence, and entity relationship views, and the complete technology stack and dataset description.

Chapter 5, Implementation and Testing, covers the practical realization of the system. It describes the frontend and backend implementation with the key engineering challenges encountered, presents the evaluation results including normalization effectiveness, topic coherence comparisons, and drift detection precision and recall, and documents the unit, integration, and system testing conducted along with the full functional requirements validation matrix.

Chapter 6, Conclusion, summarizes the key contributions of the project, reflects on the outcomes achieved relative to the stated objectives, and outlines seven specific directions for future work including language extension, real-time processing, fine-tuned embeddings, advanced drift detection, sentiment integration, cloud deployment, and corpus expansion.

CHAPTER 2

LITERATURE REVIEW

Before we began designing the UATM system, we conducted a thorough review of related work across four areas: traditional and neural topic modeling, transformer-based NLP methods, code-mixed language processing, and user behavior modeling and drift detection. This chapter presents our findings from that survey, organized by research area, and concludes by identifying the specific gaps that motivated our project.

2.1 TRADITIONAL TOPIC MODELING APPROACHES

2.1.1 Latent Dirichlet Allocation (LDA)

The most foundational work in topic modeling is Latent Dirichlet Allocation (LDA), introduced by Blei, Ng, and Jordan in 2003. LDA is a generative probabilistic model that treats each document as a mixture of latent topics, and each topic as a probability distribution over words. Given a corpus of documents, LDA learns both the per-document topic distributions and the per-topic word distributions simultaneously using variational inference or Gibbs sampling. LDA has been applied extensively to news corpora, scientific literature, and social media datasets, and remains a widely used baseline in topic modeling research.

However, LDA suffers from well-documented limitations when applied to short, noisy social media text. Social media posts are typically only a few sentences long, providing insufficient context for LDA to reliably infer topic distributions. The model relies on word co-occurrence statistics computed within individual documents, which is ineffective when documents contain very few words. Furthermore, LDA treats each word form as a distinct token, meaning it cannot recognize that semantically identical words spelled differently, as is common in Tanglish, refer to the same concept. These limitations severely reduce LDA's effectiveness on our target data.

2.1.2 Biterm Topic Model (BTM)

To address LDA's sparsity problem for short texts, Yan et al. (2013) proposed the Biterm Topic Model. Instead of computing co-occurrences within documents, BTM aggregates word-pair (biterm) statistics across the entire corpus, which compensates for the lack of within-document co-occurrence signal. BTM has shown improved performance on Twitter and other short-text corpora compared to LDA. However,

like LDA, BTM relies on surface-level word forms and cannot handle the orthographic variability inherent in Tanglish text. It also does not incorporate semantic embeddings, which means it cannot capture semantic similarity between words.

2.1.3 Non-Negative Matrix Factorization (NMF)

Non-Negative Matrix Factorization is another classical approach to topic modeling that decomposes a term-document matrix into topic-word and document-topic factor matrices with non-negative constraints. NMF tends to produce sparser and more interpretable topics than LDA on short texts. Lee and Seung (1999) demonstrated that NMF learns parts-based representations, which aligns well with the intuition of topics as coherent groupings of keywords. However, NMF shares LDA's fundamental weakness: it operates on bag-of-words representations and cannot account for semantic relationships between words or handle morphological variation.

2.2 BERTOPIC AND TRANSFORMER-BASED TOPIC MODELING

2.2.1 BERTopic

A significantly more powerful approach to topic modeling is BERTopic, introduced by Grootendorst in 2022. BERTopic replaces the bag-of-words document representation with dense semantic embeddings produced by pre-trained transformer models. The pipeline works in three stages. First, each document is encoded into a fixed-length embedding vector using a sentence transformer model. Second, UMAP is applied to reduce the high-dimensional embeddings to a lower-dimensional space while preserving global structure. Third, HDBSCAN clusters the reduced embeddings, and each cluster is assigned a topic represented by its most distinctive keywords, identified using a class-based variant of TF-IDF called c-TF-IDF.

BERTopic has several critical advantages for our use case. Because it operates on semantic embeddings rather than surface word forms, it can recognize that tokens with different spellings but similar meanings are semantically related. This is directly relevant to Tanglish, where the same word might appear in many different transliterated forms. Furthermore, HDBSCAN assigns outlier documents to a special noise cluster rather than forcing them into the nearest topic, which is more appropriate for social media corpora that contain off-topic or irrelevant posts. Grootendorst demonstrated that BERTopic achieves substantially higher coherence scores than LDA on several benchmark datasets. We selected BERTopic as the core topic modeling component of UATM based on these properties.

2.2.2 Top2Vec

Top2Vec, proposed by Angelov in 2020, takes a similar embedding-and-clustering approach to BERTopic. It uses Doc2Vec or universal sentence encoders to embed documents, reduces dimensionality with UMAP, and clusters with HDBSCAN. Topic representations are derived from the dense regions of the embedding space rather than c-TF-IDF. Top2Vec is computationally efficient and produces meaningful topics, but it offers less flexibility in model configuration and has more limited support for multilingual embeddings compared to BERTopic. For our Tanglish use case, the multilingual embedding support of BERTopic was a decisive factor.

2.2.3 Contextualized Topic Models (CTM)

Bianchi et al. (2021) proposed Contextualized Topic Models, which combine the contextual embeddings from pre-trained language models with the bag-of-words representation used by classical topic models. CTM uses a variational autoencoder architecture where the encoder takes contextual embeddings as input and the decoder reconstructs the bag-of-words representation. This approach bridges the gap between neural embeddings and probabilistic topic models. However, CTM still relies partially on the bag-of-words representation, which limits its ability to handle the extreme orthographic variability of Tanglish. We ultimately chose BERTopic over CTM for its cleaner architecture and stronger multilingual support.

2.3 CODE-MIXED AND TRANSLITERATED NLP RESEARCH

2.3.1 Language Identification in Code-Mixed Text

A foundational challenge in code-mixed NLP is language identification at the token level, meaning determining which language each word in a mixed-language utterance belongs to. Solorio and Liu (2008) were among the early researchers to systematically study code-switching in NLP, exploring part-of-speech tagging for English-Spanish code-mixed text. Subsequent work by Vyas et al. (2014) developed a language identification system for Hindi-English code-mixed social media text. These systems typically use character n-gram features, dictionary lookup, and machine learning classifiers. While language identification is a useful preprocessing step, it does not address the normalization challenge posed by transliterated Tanglish, where the same language is represented in inconsistent orthographic forms.

2.3.2 Text Normalization for Social Media

Text normalization for informal social media text has been studied extensively in the context of standard English. Han and Baldwin (2011) proposed a normalization system for English tweets that used string similarity and phonetic encoding to map non-standard word forms to their standard equivalents. Sproat and Jaitly (2017) explored sequence-to-sequence neural models for text normalization, demonstrating that neural approaches can learn normalization rules from data without hand-crafted rules. These approaches informed our design of the Tier 1 and Tier 2 normalization components, though they required significant adaptation for the Tanglish context.

2.3.3 Hindi-English Code-Mixed NLP

The most closely related body of work to ours deals with Hindi-English code-mixed text, commonly called Hinglish. Bhat et al. (2015) proposed a normalization framework for Hindi social media text that mapped transliteration variants to canonical forms using phonetic similarity measures, which directly inspired our Tier 2 phonetic normalization approach. Bhat et al. (2018) further extended this to universal dependency parsing for Hindi-English code-switching, demonstrating that proper normalization is a prerequisite for downstream NLP tasks. Kula et al. (2020) showed that multilingual pre-trained models like mBERT achieve better performance on Hinglish tasks when combined with transliteration normalization.

2.3.4 Tamil-English and Telugu NLP

Work specifically targeting South Indian code-mixed languages is considerably more limited. Swaminathan et al. (2020) addressed normalization for Tamil-English code-mixed social media text, which shares many characteristics with Tanglish due to the Dravidian language family connection and similar social media usage patterns. Their work demonstrated that phonetic-based normalization significantly improves downstream NLP task performance on Tamil-English text. For Telugu specifically, Rao et al. (2018) developed a corpus of Telugu social media text and explored basic NLP tasks, but did not address code-mixed normalization or topic modeling. To our knowledge, no prior work has addressed topic modeling specifically on Tanglish text, which represents the core research gap filled by this project.

2.4 USER MODELING AND TEMPORAL DRIFT DETECTION

2.4.1 Temporal User Interest Modeling

The idea of modeling how user interests evolve over time is well-studied in the context of recommender systems and information retrieval. Zhang et al. (2020) proposed a temporal user interest model that uses attention mechanisms to weight recent interactions more heavily when computing user embeddings for news recommendation. Their approach demonstrated that temporal modeling significantly improves recommendation quality over static user profiles. However, this work operates on standard news text and does not address the code-mixed language challenge. Lv et al. (2021) explored user interest drift detection using contrastive learning, training a model to distinguish between a user's stable long-term interests and their shifting short-term interests. These works established the conceptual foundation for our user-aware drift detection module.

2.4.2 Jensen-Shannon Divergence for Distribution Comparison

The mathematical foundation of our drift detection approach is Jensen-Shannon Divergence (JSD), introduced by Lin (1991) as a symmetrized and smoothed variant of the Kullback-Leibler Divergence. Unlike KL Divergence, JSD is symmetric, always bounded between 0 and 1, and well-defined even when one distribution has zero probability mass where the other does not. JSD has been applied in NLP for document similarity measurement, language model evaluation, and dialect identification. We adapted it as a measure of topic distribution shift between consecutive user time windows, where a high JSD value indicates significant drift in the user's interests.

2.4.3 Shannon Entropy in NLP

Shannon Entropy, introduced by Shannon (1948), measures the uncertainty or diversity of a probability distribution. In the context of topic modeling, entropy over a user's topic distribution measures how spread out the user's posts are across topics. A high entropy value indicates that a user posts about many different topics with roughly equal frequency, while a low entropy value indicates that the user predominantly posts about one or a few topics. We use entropy as a complementary signal to JSD in our drift detection module, as a sudden spike in entropy can indicate a behavioral change even when the topics themselves have not shifted dramatically.

2.5 LIMITATIONS OF EXISTING WORK

Based on our survey of the existing literature, we identified the following specific gaps that motivated the design and implementation of UATM:

2.5.1 No End-to-End Tanglish Topic Modeling System:

No existing system combines Telugu-English transliteration normalization with transformer-based topic modeling in a single end-to-end pipeline. Prior work on topic modeling of Indian social media text has either focused on Hindi-English or has not addressed the normalization challenge at all.

2.5.2 Inadequacy of LDA-Based Methods:

Traditional topic models like LDA and BTM are fundamentally inadequate for Tanglish text due to their reliance on surface word forms and their poor performance on short documents. The extreme orthographic variability of Tanglish means that the same semantic content may appear in hundreds of different surface forms, which confounds bag-of-words models.

2.5.3 Absence of User-Aware Temporal Analysis:

No existing system tracks per-user topic distributions over time for Tanglish social media content. User interest drift detection has been studied in the context of English-language recommender systems, but has not been applied to code-mixed social media text from Telugu-speaking communities.

2.5.4 Lack of Full-Stack Deployment:

Prior NLP research on code-mixed Indian languages has produced academic models and datasets but has not delivered deployable full-stack applications with user-facing dashboards. UATM addresses this by combining the NLP pipeline with a React-based frontend and FastAPI backend, making the system accessible to non-technical users.

2.5.5 Telugu-Specific Research Gap:

While Hinglish and Tamil-English code-mixed NLP have received some research attention, Telugu-English code-mixing remains significantly understudied. Given the scale of Telugu social media usage across Andhra Pradesh and Telangana, this represents a meaningful gap in the NLP literature that this project directly addresses.

CHAPTER 3
REQUIREMENT ANALYSIS

Before we started building the system, we spent time clearly defining what the system needed to do, both from a functional perspective and from a quality and reliability perspective. We consulted with our project guide multiple times during this phase to ensure our requirements were realistic, well-scoped, and aligned with the goals of the project. We also looked at how similar systems in the literature handled requirements, particularly the way ontology-driven knowledge systems define constraint requirements. This chapter documents the requirements we identified and the reasoning behind them.

3.1 FUNCTIONAL REQUIREMENTS

Functional requirements define the specific behaviors and capabilities that the system must provide. We organized our functional requirements into two groups: those relating to the core NLP pipeline (normalization, topic modeling, and drift detection) and those relating to the web application (authentication, APIs, and dashboards). Each requirement is assigned a unique identifier for traceability.

Requirement ID	**Description**
FR-01	The system shall accept raw Tanglish text input and produce a normalized form of that text as output through a three-tier pipeline.
FR-02	The system shall perform Tier 1 lexical normalization including tokenization, lowercasing, stopword removal, and dictionary-based correction of common Tanglish spelling variants.
FR-03	The system shall perform Tier 2 phonetic normalization using Soundex and Metaphone encoding to group orthographic variants of the same word.

FR-04	The system shall perform Tier 3 contextual normalization using a pre-trained multilingual masked language model to resolve ambiguous tokens based on surrounding context.
FR-05	The system shall extract topics from normalized text using BERTopic with a multilingual sentence transformer embedding model.
FR-06	The system shall assign each post to a topic and store this assignment in the database associated with the submitting user.
FR-07	The system shall compute and store per-user topic probability distributions at configurable time intervals.
FR-08	The system shall compute Shannon Entropy over each user's topic distribution to quantify interest diversity.
FR-09	The system shall compute Jensen-Shannon Divergence between a user's current and previous topic distributions to quantify interest drift.
FR-10	The system shall flag users whose JSD exceeds a configurable threshold and generate a drift alert record in the database.
FR-11	The system shall support user registration, authentication via JWT tokens, and role-based access control with at least two roles: Administrator and Regular User.
FR-12	The system shall provide an Administrator dashboard showing global topic distribution across all users, a list of drift-flagged users with their JSD scores, and controls for system configuration.

FR-13	The system shall provide a Regular User dashboard showing the user's personal topic distribution, entropy history, JSD drift scores, and received drift alerts.
FR-14	The system shall expose a RESTful API that the frontend uses to interact with all backend functionality.

3.2 NON-FUNCTIONAL REQUIREMENTS

Beyond what the system does, we identified requirements for how well it should do it. These non-functional requirements governed key design decisions throughout the project.

3.2.1 Performance

The system should respond to single-user topic query requests within 3 seconds under normal operating conditions. Full corpus re-modeling, which involves re-embedding and re-clustering all stored posts, may take longer but should complete within 15 minutes for a dataset of 50,000 posts on the minimum specified hardware. Drift computation for a single user should complete within 500 milliseconds.

3.2.2 Usability

The dashboard should be intuitive enough that a non-technical user, such as a social media analyst or researcher with no programming background, can understand their topic distribution and drift indicators without any specialized training. Topic labels should be presented using their top keywords rather than numeric cluster identifiers. Drift alerts should be accompanied by a brief natural-language explanation of what the alert means.

3.2.3 Security

All API endpoints that return user-specific data must require a valid JWT authentication token. User passwords must be stored as salted hashes and never in plaintext. Input text must be sanitized before processing to prevent SQL injection and cross-site scripting attacks. The system must enforce role-based access control, ensuring that regular users can only access their own data and that administrator endpoints are inaccessible to regular users.

3.2.4 Reliability and Robustness

The system must handle edge cases gracefully without crashing or returning uninformative errors. Edge cases include empty text inputs, posts consisting of a single word, posts containing no recognizable Tanglish content, users who have submitted fewer posts than the minimum cluster size for HDBSCAN, and database connection failures. All such cases must return informative error responses with appropriate HTTP status codes.

3.2.5 Maintainability and Modularity

The codebase must be organized into clearly separated modules corresponding to the major components of the system: the normalization pipeline, the BERTopic integration, the drift detection module, the FastAPI application, and the React frontend. Each module should have a well-defined interface and minimal coupling with other modules, so that individual components can be updated or replaced without affecting the rest of the system.

3.3 SOFTWARE REQUIREMENTS

The following table lists the software components required to build and run the UATM system, along with the specific versions and purposes of each component.

Component	**Specification and Purpose**
Operating System	Windows 10/11 or Ubuntu 20.04+ – Development and deployment environment
Python	Python 3.10+ – Primary language for backend and NLP pipeline
JavaScript	ES6+ – Primary language for React frontend
FastAPI	Version 0.100+ – Backend web framework for REST API

React	Version 18+ – Frontend framework for user dashboards
Uvicorn	ASGI server for running FastAPI in production
BERTopic	Version 0.15+ – Core topic modeling library
sentence-transformers	paraphrase-multilingual-MiniLM-L12-v2 – Multilingual document embeddings
UMAP-learn	Dimensionality reduction for BERTopic embedding space
HDBSCAN	Density-based clustering for topic discovery
SQLAlchemy	ORM for database interactions
SQLite	Development database
PostgreSQL	Production database
python-jose	JWT token generation and validation
passlib	Password hashing with bcrypt
pytest	Unit and integration testing framework
Tailwind CSS	Utility-first CSS framework for frontend styling
Recharts	React charting library for data visualizations
Git / GitHub	Version control and remote repository hosting

3.4 HARDWARE REQUIREMENTS

The following hardware specifications represent the minimum configuration required to run the UATM system. We recommend the higher specifications for comfortable development and testing with large datasets.

Component	**Minimum / Recommended**
Processor	Intel Core i5 (8th Gen) or equivalent / Intel Core i7 or AMD Ryzen 7
RAM	8 GB minimum / 16 GB recommended (BERTopic model loads ~900 MB into memory)
Storage	20 GB free disk space for application, models, and dataset
GPU	Optional – NVIDIA CUDA-compatible GPU significantly accelerates embedding generation
Operating System	Windows 10/11, Ubuntu 20.04+, or macOS 12+
Network	Required for initial download of pre-trained sentence transformer models
Browser	Any modern browser (Chrome, Firefox, Edge) for accessing the React dashboard

3.5 USER REQUIREMENTS

We identified two categories of users for the UATM system: Administrators, who manage the system and monitor global trends, and Regular Users, who interact with the system to submit content and explore their personal topic history. The requirements for each role are described below.

3.5.1 Administrator Requirements

The Administrator is a technically proficient user, such as a system manager or researcher, who is responsible for overseeing the operation of the UATM platform. The administrator's requirements are as follows:

- The administrator must be able to log in using a privileged account and access the admin dashboard, which is not visible to regular users.
- The administrator must be able to view a global summary of all topics discovered across the entire corpus, including the number of posts assigned to each topic and the top keywords for each topic.
- The administrator must be able to view a list of all users flagged for significant topic drift in the most recent time window, along with each user's JSD score and the direction of drift.
- The administrator must be able to adjust the JSD drift detection threshold through the dashboard without modifying any code.
- The administrator must be able to trigger a full corpus re-modeling run on demand through the dashboard.
- The administrator must be able to manage user accounts, including viewing all registered users, deactivating accounts, and resetting passwords.
- The administrator must be able to view system health metrics such as the total number of posts processed, the number of topics in the current model, and the last re-modeling timestamp.

3.5.2 Regular User Requirements

The Regular User is a non-technical end user who submits Tanglish text content and explores their personal topic history and drift indicators. The regular user's requirements are as follows:

- A regular user must be able to register for an account and log in using their credentials.
- A regular user must be able to submit Tanglish text through the dashboard interface and receive a topic assignment in response.

- A regular user must be able to view their current topic distribution as a visual chart, showing the proportion of their posts assigned to each discovered topic.
- A regular user must be able to see how their topic distribution has evolved over time, presented as a time-series visualization.
- A regular user must be able to view the top keywords associated with each of their topics, so they can understand what each topic represents.
- A regular user must receive a drift alert notification on their dashboard if their JSD score exceeds the configured threshold, with a plain-language explanation of what the alert means.
- A regular user must be able to view their Shannon Entropy score and understand intuitively whether their posting behavior is diverse or focused.

CHAPTER 4
SYSTEM DESIGN

This chapter presents a detailed description of the design of the UATM system. We begin with the overall system architecture, then describe each major component in detail, including the three-tier normalization pipeline, the BERTopic integration, and the drift detection module. We also present the UML diagrams developed during the design phase and describe the dataset and technology stack used in the implementation.

4.1 PROPOSED SYSTEM ARCHITECTURE

The UATM system is built on a three-layer client-server architecture. At the top, the Presentation Layer consists of a React-based web frontend that renders role-specific dashboards for administrators and regular users. In the middle, the Application Logic Layer consists of a FastAPI-based backend that handles HTTP requests, orchestrates the NLP pipeline, enforces authentication and authorization, and computes analytics. At the bottom, the Data Layer consists of a relational database for storing user accounts, post data, topic assignments, topic distributions, and drift records, along with a file-based store for the persisted BERTopic model.

The key design principle of the UATM architecture is the separation of concerns between the NLP pipeline and the web application. The normalization pipeline, BERTopic model, and drift detection module are implemented as independent Python modules that the FastAPI backend calls as services. This separation means that the NLP components can be updated, replaced, or reconfigured without modifying the web application code, and vice versa.

The data flow through the system proceeds as follows. When a user submits text through the React frontend, the frontend sends an authenticated HTTP POST request to the /analyze endpoint on the FastAPI backend. The backend first validates the JWT token to confirm the user's identity and role. The text is then passed through the three-tier normalization pipeline, which produces a cleaned, canonical version of the input. The normalized text is then embedded using the sentence transformer model and passed to the BERTopic model for topic assignment. The resulting topic ID is stored in the database along with a timestamp and the user's ID. After the post is stored, the system recomputes the user's topic distribution and drift metrics. If the JSD exceeds the configured threshold, a drift alert is written to the database. The

response returned to the frontend includes the assigned topic, its top keywords, the user's updated entropy score, and any new drift alert.

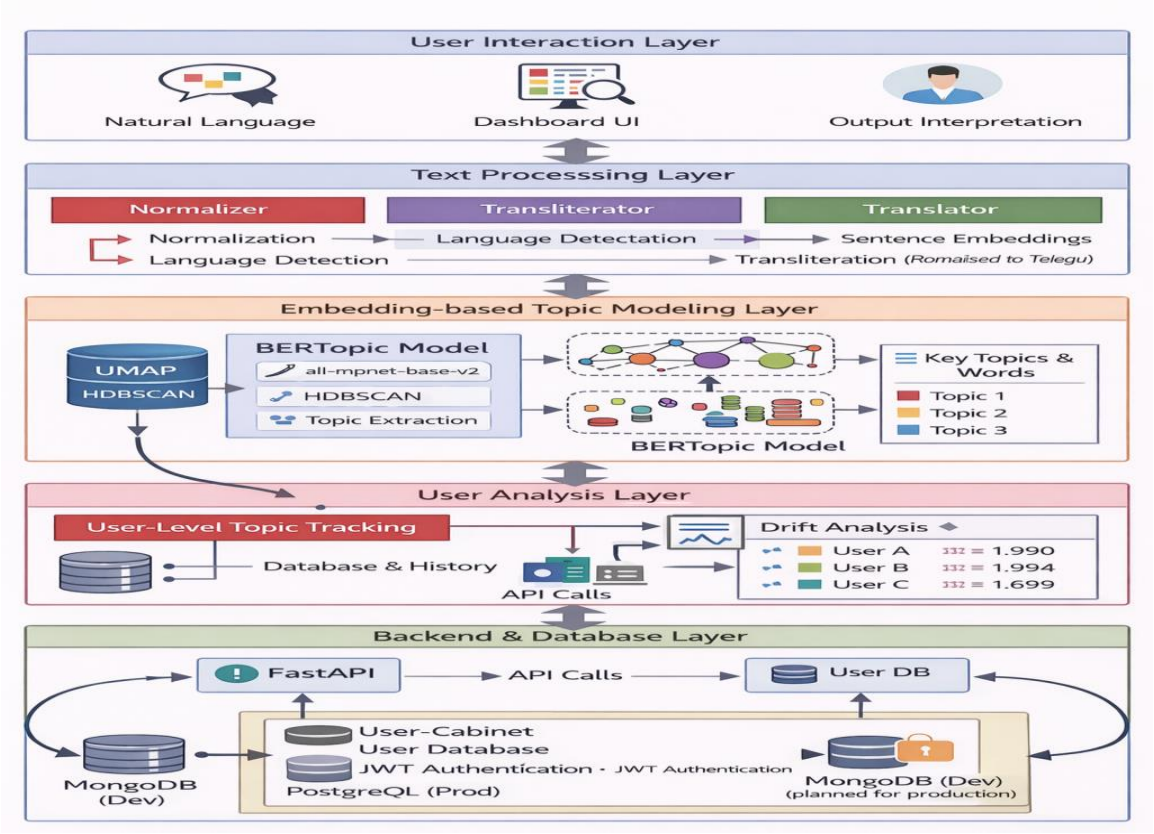

Figure 4.1: Overall System Architecture of UATM

4.2 NORMALIZATION PIPELINE DESIGN

The normalization pipeline is the most novel and domain-specific component of the UATM system. It is designed as a three-tier sequential process, where each tier addresses a distinct category of noise or variability in Tanglish text. The tiers are applied in order, and the output of each tier serves as the input to the next.

4.2.1 Tier 1 – Lexical Normalization

The first tier performs basic text cleaning and dictionary-based lexical correction. The processing steps in Tier 1 are applied in the following sequence:

1. Lowercasing: All text is converted to lowercase to eliminate case-based variation. For example, 'Bagundi', 'BAGUNDI', and 'bagundi' are collapsed to a single form.
2. Tokenization: The text is split into tokens using whitespace and punctuation as delimiters. We use a custom tokenizer rather than a standard one to correctly handle mixed-script tokens that may contain both ASCII and Unicode characters.
3. Punctuation and noise removal: Punctuation marks, emoji characters, URLs, and @mentions are removed. Repeated characters are compressed, so 'superrrrr' becomes 'super' and 'hahahahaha' becomes 'haha'.
4. Stopword removal: A combined list of Telugu Romanized stopwords (such as 'nenu', 'mee', 'adi', 'idi') and English stopwords is applied to remove high-frequency, low-information tokens.
5. Dictionary-based correction: Each remaining token is looked up in a custom Tanglish normalization dictionary of approximately 800 entries. If a match is found, the token is replaced with its canonical form.

Compiling the Tier 1 dictionary was one of the most labour-intensive parts of the project. We manually reviewed a random sample of 2,000 Tanglish posts and identified the most frequently occurring non-standard word forms. For each such form, we determined the canonical representation and added it to the dictionary. The dictionary covers common Tanglish words, abbreviations, numerals used as words (such as 'u' for 'you'), and the most frequent transliteration variants.

4.2.2 Tier 2 – Phonetic Normalization

The second tier handles orthographic variants that the dictionary cannot capture because they are phonetically plausible but not listed as dictionary entries. Tanglish text often contains words that sound identical when spoken but are spelled in multiple different ways by different users. For example, the Telugu word meaning 'good' might appear as 'manchidi', 'manchgaa', 'manchidi', 'manchidu', or 'manchii' across different posts. These variants would not all be in the Tier 1 dictionary, but they are clearly related.

Tier 2 addresses this using phonetic encoding. Each remaining token after Tier 1 is encoded using both the Soundex algorithm and the Metaphone algorithm. Soundex produces a four-character code that captures the rough phonetic structure of an English word by encoding consonants and ignoring vowels. Metaphone is a more sophisticated algorithm that encodes words based on English phonetic rules, handling digraphs and consonant clusters more accurately than Soundex.

Tokens that share the same Soundex code and the same Metaphone code are placed into the same equivalence class. For each equivalence class, the most frequently occurring member in our reference corpus is selected as the canonical form, and all other members of the class are mapped to it. This allows the system to generalize normalization beyond the explicit dictionary entries in Tier 1.

4.2.3 Tier 3 – Contextual Normalization

After Tiers 1 and 2, some tokens may still be ambiguous. A token might be resolvable to two different canonical forms depending on the context in which it appears. For example, the Tanglish token 'bank' could refer to a financial institution or to the bank of a river, and the correct interpretation depends on the surrounding words.

Tier 3 resolves such ambiguities using a pre-trained multilingual masked language model, specifically mBERT (multilingual BERT). For each token flagged as ambiguous by Tier 2, we construct a masked version of the sentence by replacing that token with the BERT [MASK] token. We then query the model for its top predictions for the masked position. The prediction that best matches one of the candidate canonical forms is selected.

Tier 3 is the most computationally expensive stage of the normalization pipeline. To keep processing times acceptable, we apply Tier 3 only to tokens that are flagged as ambiguous, which in practice is a small fraction of all tokens. Tokens unambiguously resolved by Tiers 1 or 2 bypass Tier 3 entirely. On our evaluation dataset, approximately 8% of tokens required Tier 3 processing.

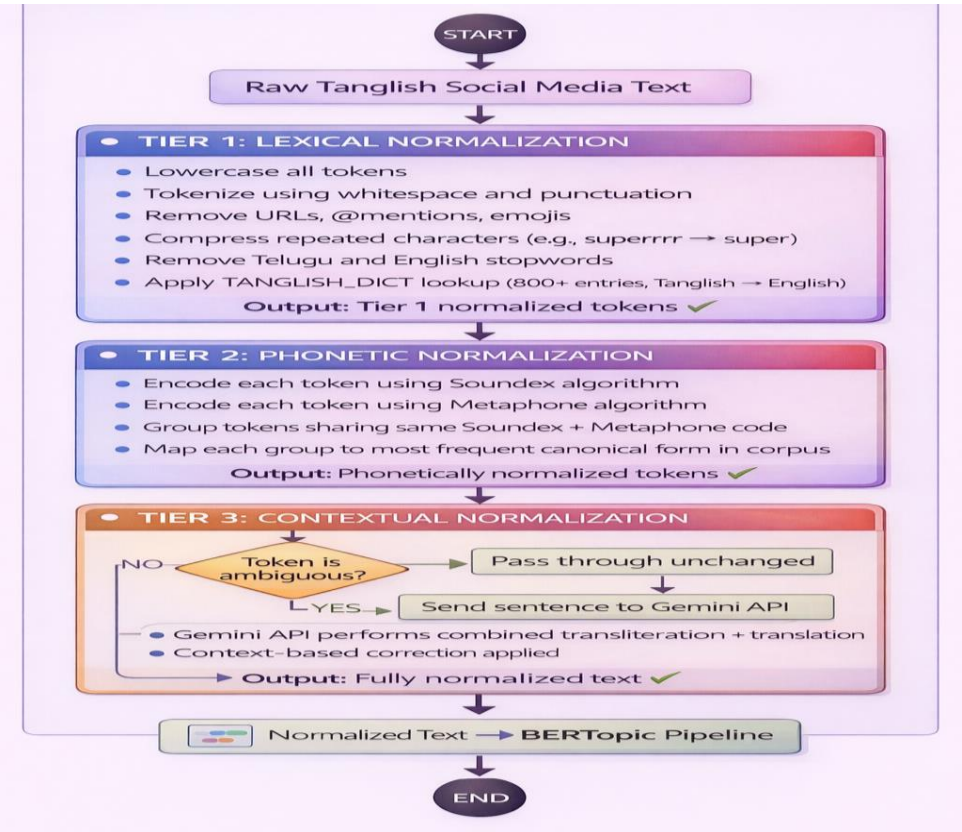

Figure 4.2: Three-Tier Tanglish Normalization Pipeline

4.3 BERTOPIC INTEGRATION DESIGN

After normalization, the processed text is passed to BERTopic for topic extraction. The BERTopic integration in UATM consists of three sub-components: the embedding model, the dimensionality reduction stage, and the clustering and topic representation stage.

4.3.1 Embedding Model Selection

We selected the paraphrase-multilingual-MiniLM-L12-v2 model from the sentence-transformers library as our embedding model. This model was chosen for several reasons. It supports over 50 languages including Telugu, having been trained on multilingual parallel corpora that include South Asian languages. It produces 384-dimensional embeddings, which is large enough to capture semantic nuance while remaining computationally tractable. It was specifically designed for semantic similarity tasks and produces well-calibrated embeddings where semantically similar documents are close in the embedding space. Compared to English-only models like all-MiniLM-L6-v2, the multilingual model produces

substantially better embeddings for Tanglish text because it has seen similar code-mixed patterns during training.

4.3.2 Dimensionality Reduction with UMAP

The 384-dimensional embeddings produced by the sentence transformer are too high-dimensional for HDBSCAN to cluster effectively due to the curse of dimensionality. We use UMAP to reduce the embedding dimensionality to 5 dimensions before clustering. We chose 5 dimensions as a compromise between preserving the global structure of the embedding space and providing a low enough dimensionality for HDBSCAN to identify tight, meaningful clusters. The UMAP parameters are configured with n_neighbors=15, which balances local and global structure preservation, and min_dist=0.0, which allows UMAP to pack points tightly together and helps HDBSCAN form cleaner clusters.

4.3.3 Clustering and Topic Representation

HDBSCAN is applied to the UMAP-reduced embeddings to identify topic clusters. HDBSCAN is a density-based clustering algorithm that does not require the number of clusters to be specified in advance and naturally handles noise points by assigning them to a special cluster labeled -1. We configure HDBSCAN with a minimum cluster size of 10, meaning that any group of fewer than 10 documents is not treated as a topic and its members are instead labeled as noise. Documents labeled as noise are subsequently assigned to the nearest topic based on cosine similarity in the original 384-dimensional embedding space.

After clustering, the topic keywords are extracted using c-TF-IDF. For each topic cluster, c-TF-IDF computes a modified TF-IDF score that treats all documents in a topic as a single concatenated document and computes IDF against the rest of the corpus. This produces keywords that are highly representative of a specific topic relative to all other topics, rather than simply the most frequent words overall. The top 10 keywords per topic are stored in the database and displayed in the user dashboards.

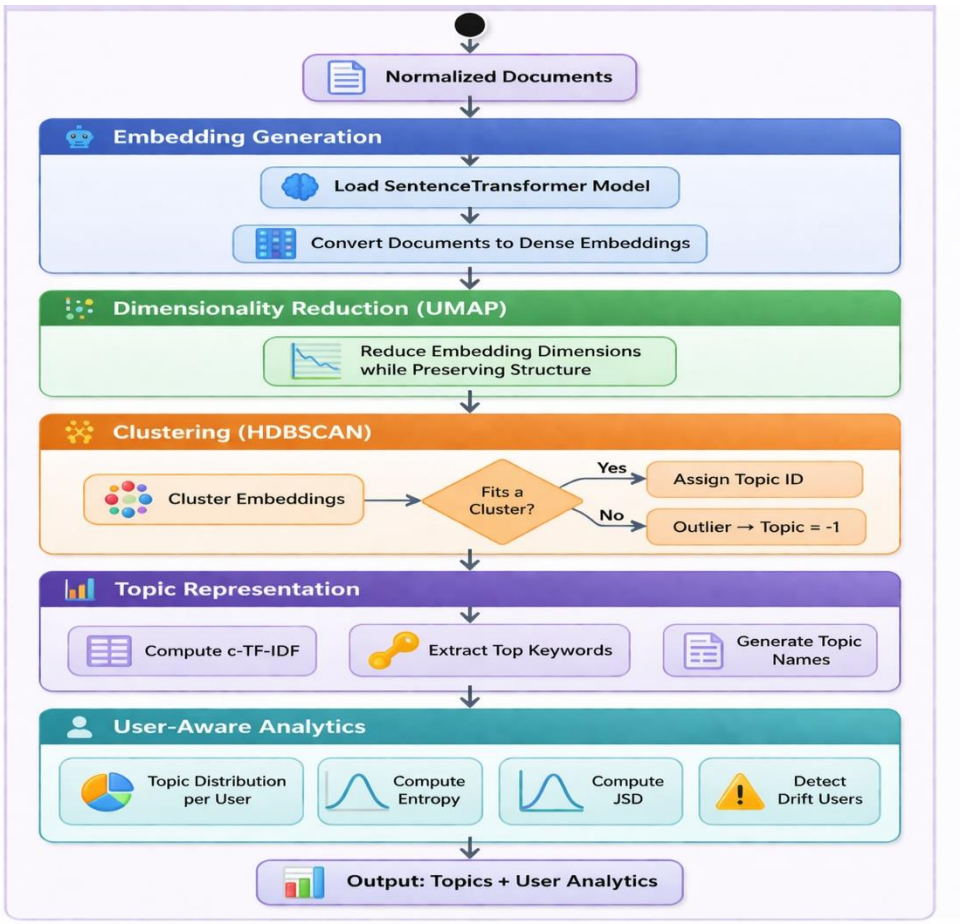

Figure 4.3: BERTopic Workflow

4.4 USER-AWARE DRIFT DETECTION DESIGN

The drift detection module is the component that makes UATM user-aware. It operates after topic assignment and is responsible for computing per-user topic distributions, measuring entropy, computing JSD between consecutive distributions, and generating drift alerts. The module is designed to be stateless: it reads topic assignment records from the database and writes drift records back, with no internal state that persists between runs.

4.4.1 Topic Distribution Computation

After each batch of posts is processed, the drift detection module computes a topic probability distribution for each user. The distribution is computed over a sliding time window of configurable length (default: one week). For each user u and each topic k, the probability P(k|u) is defined as the fraction of user u's

posts within the current time window that are assigned to topic k. This produces a probability vector over all topics for each user, which is stored in the TopicDistribution table in the database.

4.4.2 Shannon Entropy Computation

Shannon Entropy H(P) over a user's topic distribution P is computed as:

H(P) = - sum over k of [P(k|u) * log2(P(k|u))]

A user who posts about many topics equally will have entropy approaching log2(K) where K is the total number of topics. A user who posts exclusively about one topic will have entropy of 0. We display each user's entropy score on their dashboard and track it over time. A sudden increase in entropy can indicate that the user has started posting about a wider range of topics, while a sudden decrease can indicate that their interests have become more focused.

4.4.3 Jensen-Shannon Divergence for Drift Detection

To quantify how much a user's topic distribution has shifted between two consecutive time windows, we compute the Jensen-Shannon Divergence between the two distributions. For two distributions P (current window) and Q (previous window), the JSD is defined as:

JSD(P || Q) = (1/2) * KL(P || M) + (1/2) * KL(Q || M), where M = (P + Q) / 2

Here KL denotes the Kullback-Leibler Divergence. JSD is symmetric, bounded between 0 and 1 (when using log base 2), and always well-defined even when one distribution has zero probability mass where the other does not. A JSD of 0 indicates that the two distributions are identical, while a JSD of 1 indicates they are maximally different. When a user's JSD exceeds the configured threshold (default: 0.30), the system generates a drift alert record in the database associated with that user and time window.

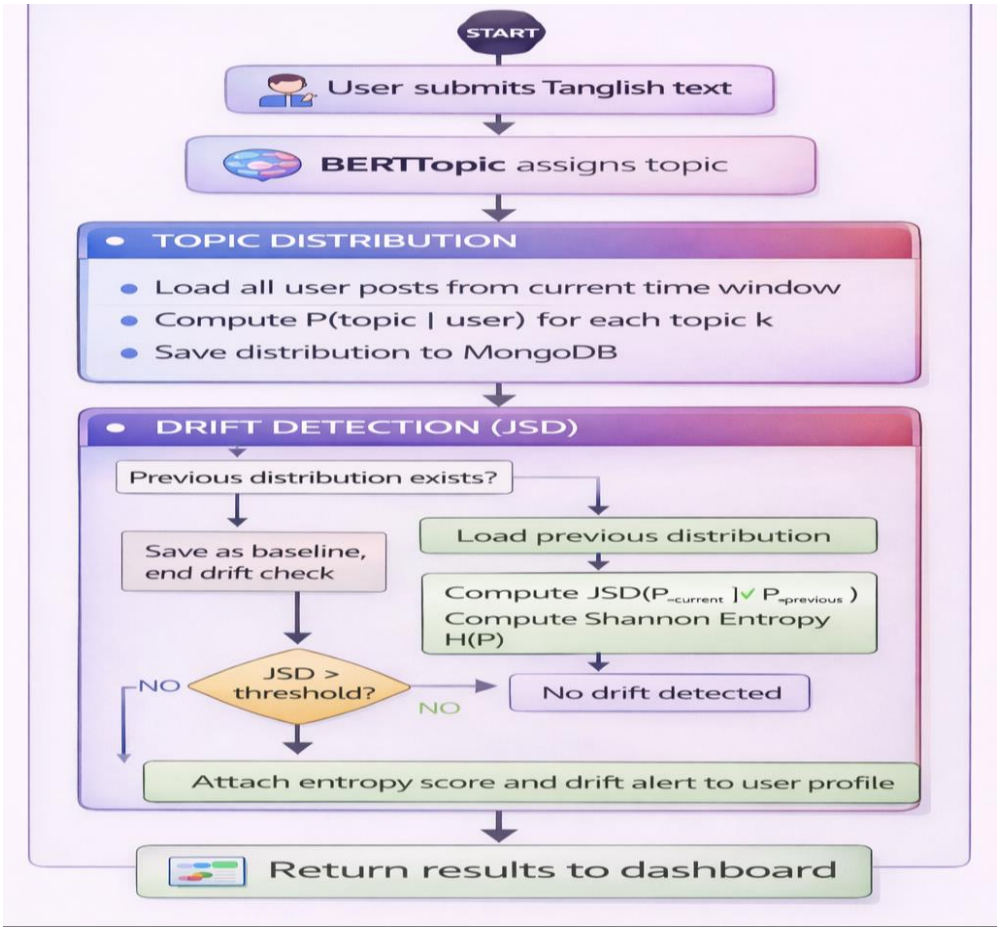

Figure 4.4: User-Aware Drift Detection Flow

4.5 UML DIAGRAMS

4.5.1 Use Case Diagram

The use case diagram shows the two main actors in the system: the Administrator and the Regular User. The Administrator can perform the following use cases: Login, View Global Topic Summary, View Drift-Flagged Users, Adjust JSD Threshold, Trigger Re-Modeling, and Manage User Accounts. The Regular User can perform the following use cases: Register, Login, Submit Text, View Personal Topic Dashboard, View Topic History, and Receive Drift Alert. Both actors share the Login use case, which is included by all other authenticated use cases.

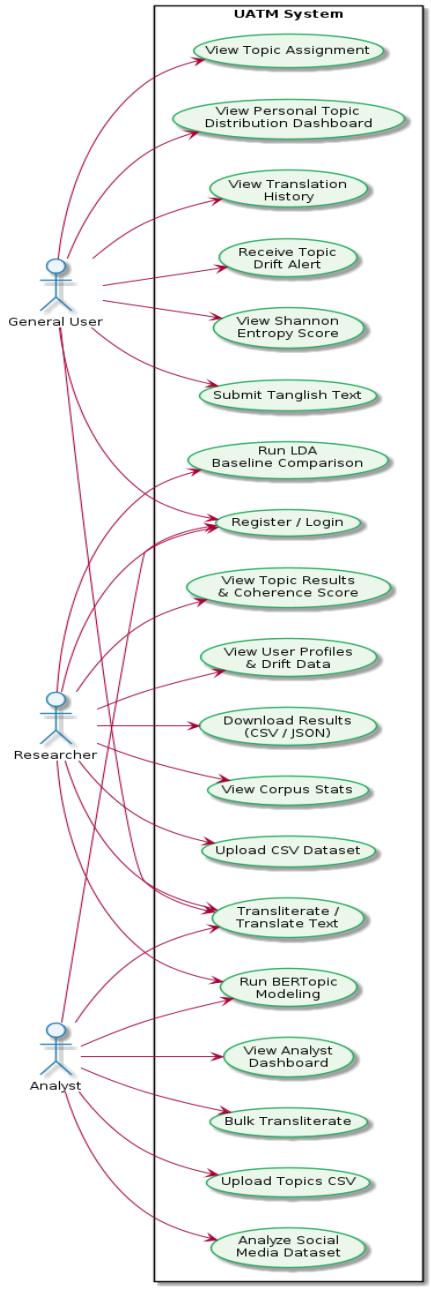

Figure 4.5: Use Case Diagram

4.5.2 Activity Diagram

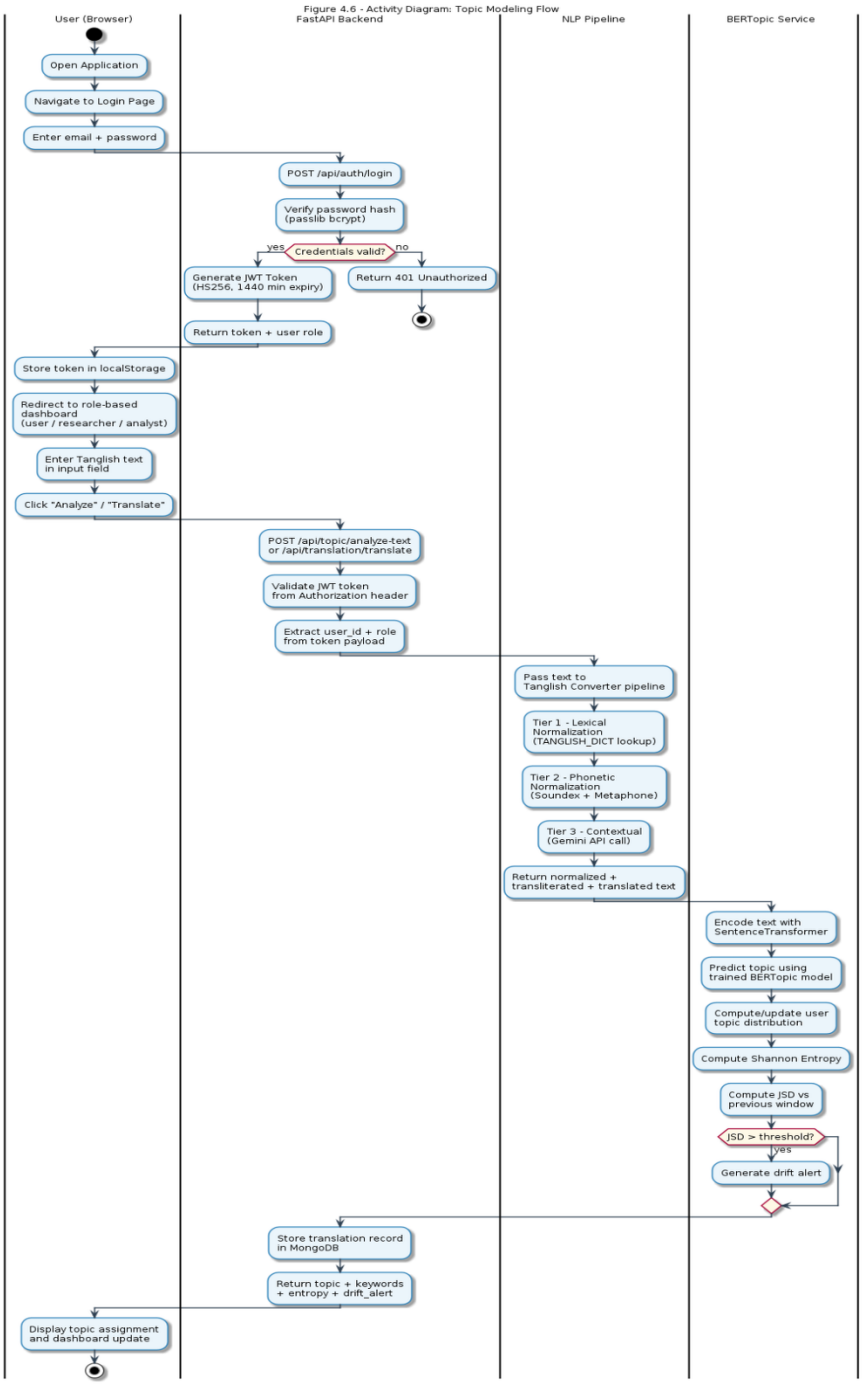

Figure 4.6: Activity Diagram

The activity diagram illustrates the sequential flow of the topic modeling process from the moment a user submits text to the moment the system returns a topic assignment and drift update. The flow begins with text submission, proceeds through the three normalization tiers, continues with embedding generation and

BERTopic topic assignment, moves to database storage and distribution computation, and concludes with drift alert generation if applicable.

4.5.3 Sequence Diagram

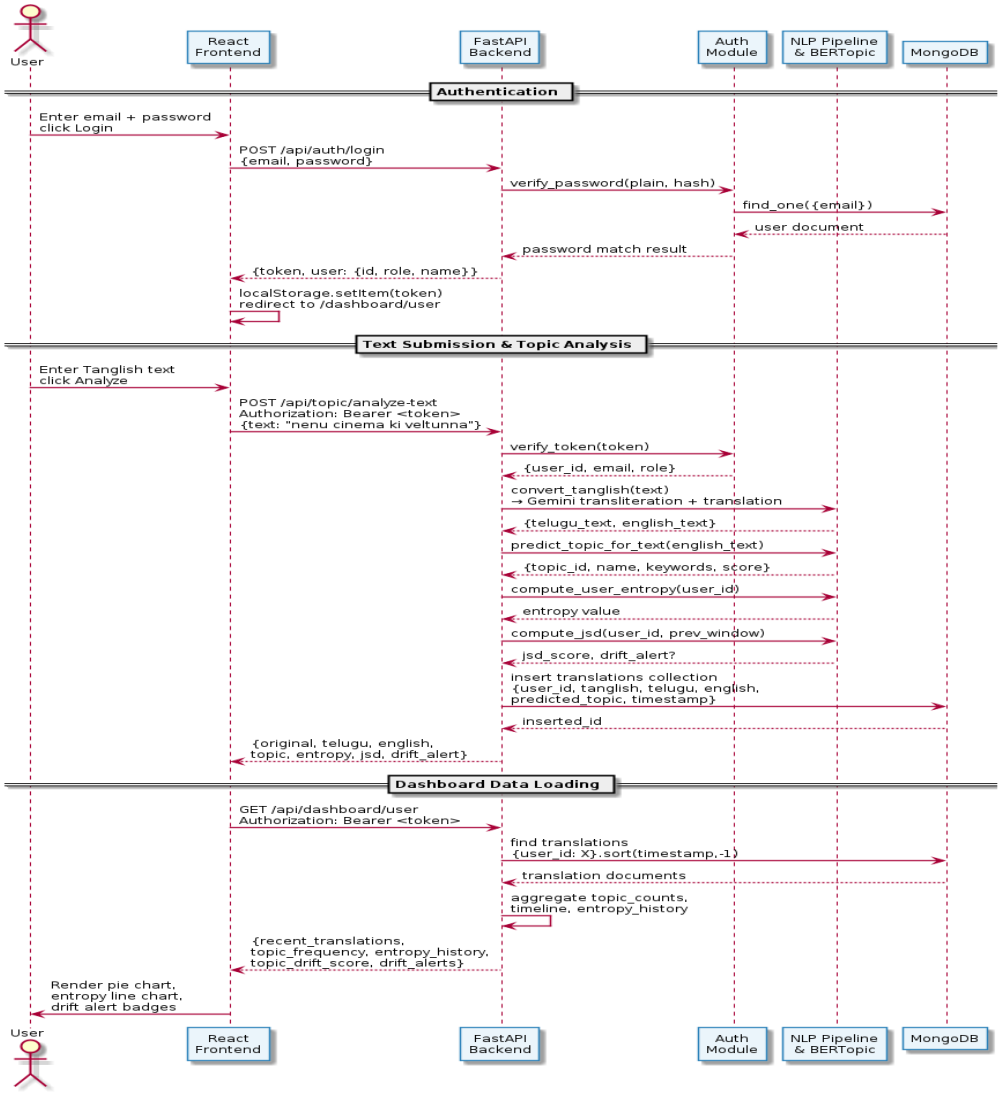

Figure 4.7: Sequence Diagram

The sequence diagram shows the interactions between the React frontend, the FastAPI backend, the NLP pipeline module, the drift detection module, and the database during a typical user session. It illustrates the message sequence for user login, text submission, topic retrieval, and dashboard rendering.

4.5.4 Entity Relationship Diagram

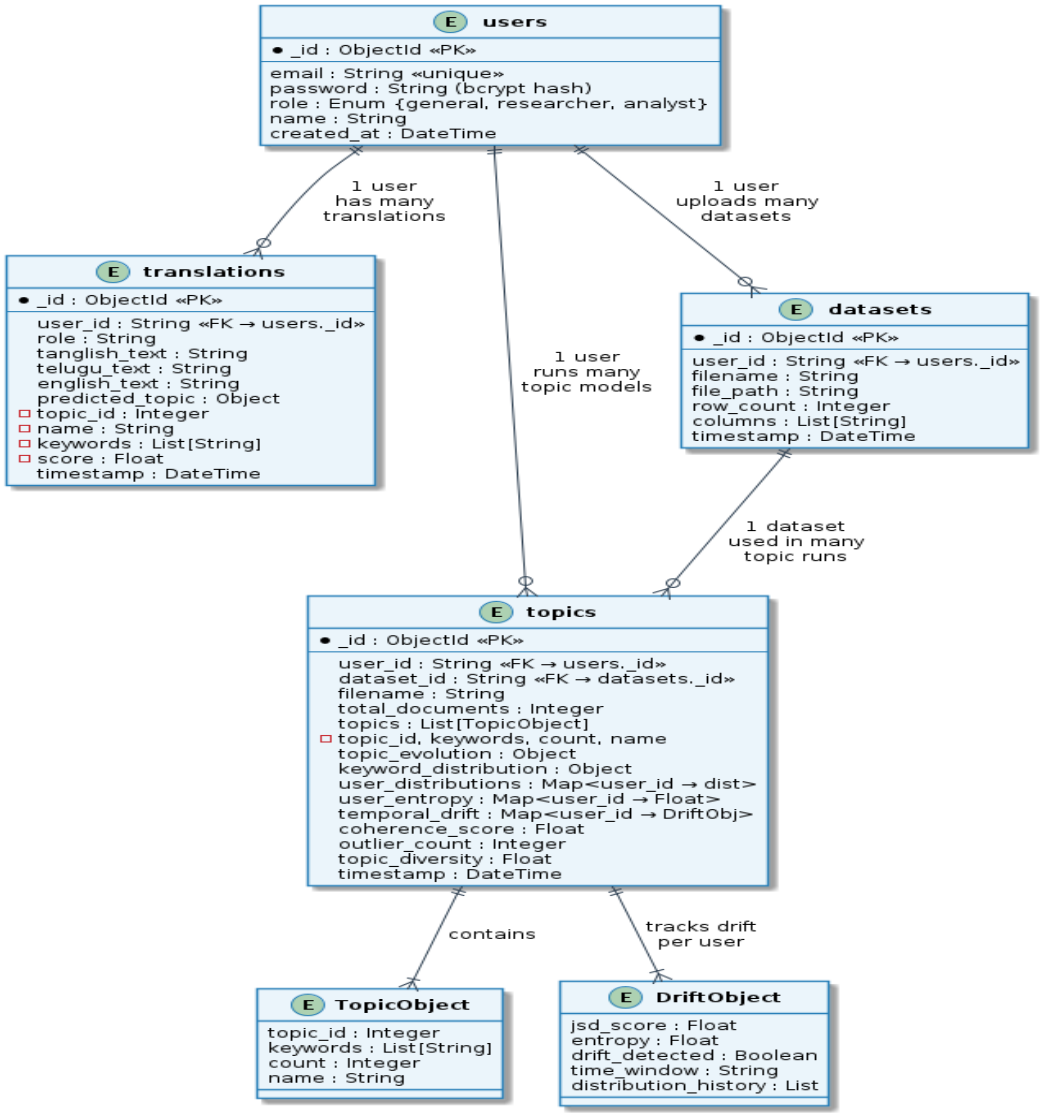

Figure 4.8: Entity Relationship Diagram

The ER diagram presents the five main entities in the UATM database and their relationships. The User entity has a one-to-many relationship with the Post entity. Each Post has a many-to-one relationship with the Topic entity. Each User has a one-to-many relationship with the TopicDistribution entity, where each TopicDistribution record corresponds to one time window. Each User also has a one-to-many relationship with the DriftRecord entity. The Topic entity has a one-to-many relationship with both Post and TopicDistribution.

4.5.5 Class Diagram

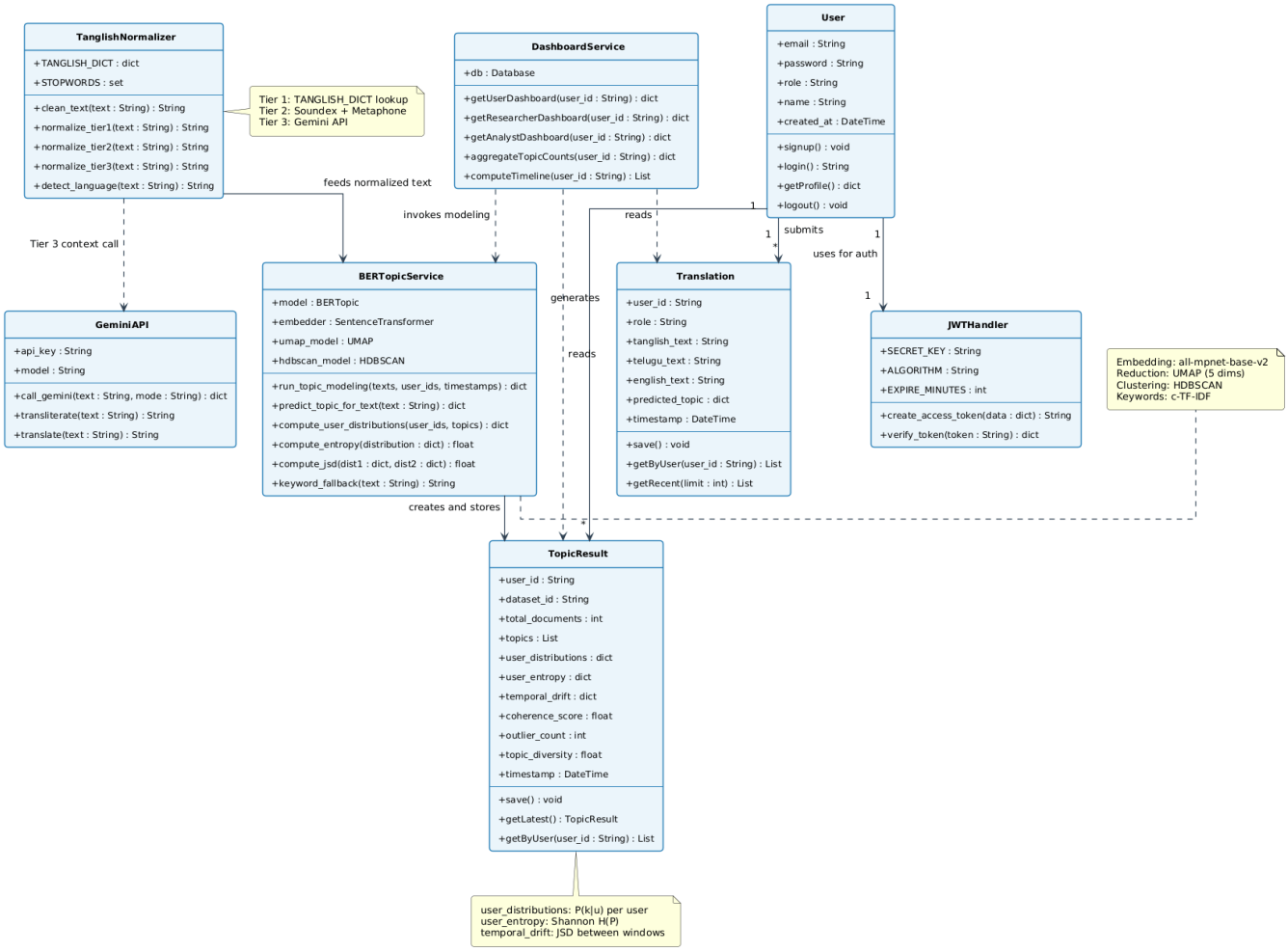

Figure 4.9 Class Diagram

This class diagram represents the architecture of a user-aware topic modeling system using BERTopic. It shows how core components like TanglishNormalizer, BERTopicService, and GeminiAPI collaborate to process, normalize, and analyze Tanglish text. The system manages user data, authentication via JWTHandler, and stores results such as topics, entropy, and drift in TopicResult. It highlights the interaction between frontend services, backend logic, and NLP pipelines for end-to-end text analysis.

4.6 DATASET AND TECHNOLOGY STACK

4.6.1 Dataset

The UATM system was trained and evaluated using a dataset of Tanglish social media posts collected from publicly available Twitter/X archives and YouTube comment sections. The dataset contains approximately 50,000 posts spanning multiple topic domains. The topic domains covered include Telugu cinema and entertainment, cricket and sports, state and national politics, food and cooking, technology and mobile phones, education and examinations, religion and festivals, and general casual conversation.

Posts were collected by searching for both Telugu keywords in native script and common Tanglish terms associated with each domain. The collection process used public APIs with rate limiting to ensure compliance with platform terms of service. All personally identifiable information, including usernames and user IDs, was anonymized before the dataset was used for development or evaluation. Posts shorter than 5 tokens after normalization were excluded from the corpus, as they were found to be too short for reliable topic assignment.

4.6.2 Technology Stack

The following table summarizes the complete technology stack used in the UATM system.

Layer	**Technology**	**Purpose**
Frontend UI	React 18, Tailwind CSS	Component-based UI, responsive styling
Frontend Charts	Recharts	Topic distribution and entropy visualizations
Backend Framework	FastAPI, Python 3.10	REST API, dependency injection, ASGI routing
ASGI Server	Uvicorn	Production-grade server for FastAPI

NLP – Embeddings	sentence-transformers (HuggingFace)	Multilingual semantic text embeddings
NLP – Topic Modeling	BERTopic 0.15	Topic extraction using embeddings and clustering
NLP – Dim. Reduction	UMAP-learn	Embedding space dimensionality reduction
NLP – Clustering	HDBSCAN	Density-based topic cluster formation
NLP – Contextual	mBERT (HuggingFace Transformers)	Tier 3 contextual ambiguity resolution
Database ORM	SQLAlchemy	Database interaction and schema management
Database (Dev)	SQLite	Lightweight local development database
Authentication	python-jose, passlib, bcrypt	JWT tokens, password hashing
Testing	pytest, FastAPI TestClient	Unit and integration tests
Version Control	Git, GitHub	Source code management and collaboration

CHAPTER 5

IMPLEMENTATION

This chapter describes the actual implementation of the UATM system, presents the results obtained during evaluation, and documents the testing and validation process. The implementation was carried out over approximately four months. The most significant time was spent on building and refining the normalization pipeline, configuring BERTopic for the Tanglish domain, and developing the React dashboards. We followed an iterative development approach, regularly testing each component independently before integrating it into the full system.

5.1 FRONTEND IMPLEMENTATION

The frontend was built using React 18 with functional components and React Hooks for state management. Tailwind CSS was used for styling, and Recharts was used for all data visualizations. The frontend communicates with the FastAPI backend through a centralized API client module that handles authentication headers, request formatting, and error handling. We implemented two distinct dashboard views corresponding to the two user roles.

5.1.1 Admin Dashboard

The administrator dashboard is the central control interface for managing the UATM system. It is organized into four panels. The first panel displays a horizontal bar chart showing the size of each discovered topic, where the bar length represents the number of posts assigned to that topic and the bar label shows the topic's top three keywords. This gives the administrator an immediate visual overview of the dominant topics in the corpus.

The second panel displays a table of users flagged for drift in the most recent time window. Each row shows the user's anonymized ID, their JSD score, and the time of detection. The administrator can click on any row to expand it and view that user's full topic distribution history. The third panel provides system configuration controls, including a slider for adjusting the JSD drift threshold and a button for triggering a full re-modeling run. The fourth panel shows system health metrics including the total post count, number of topics, and last re-modeling timestamp.

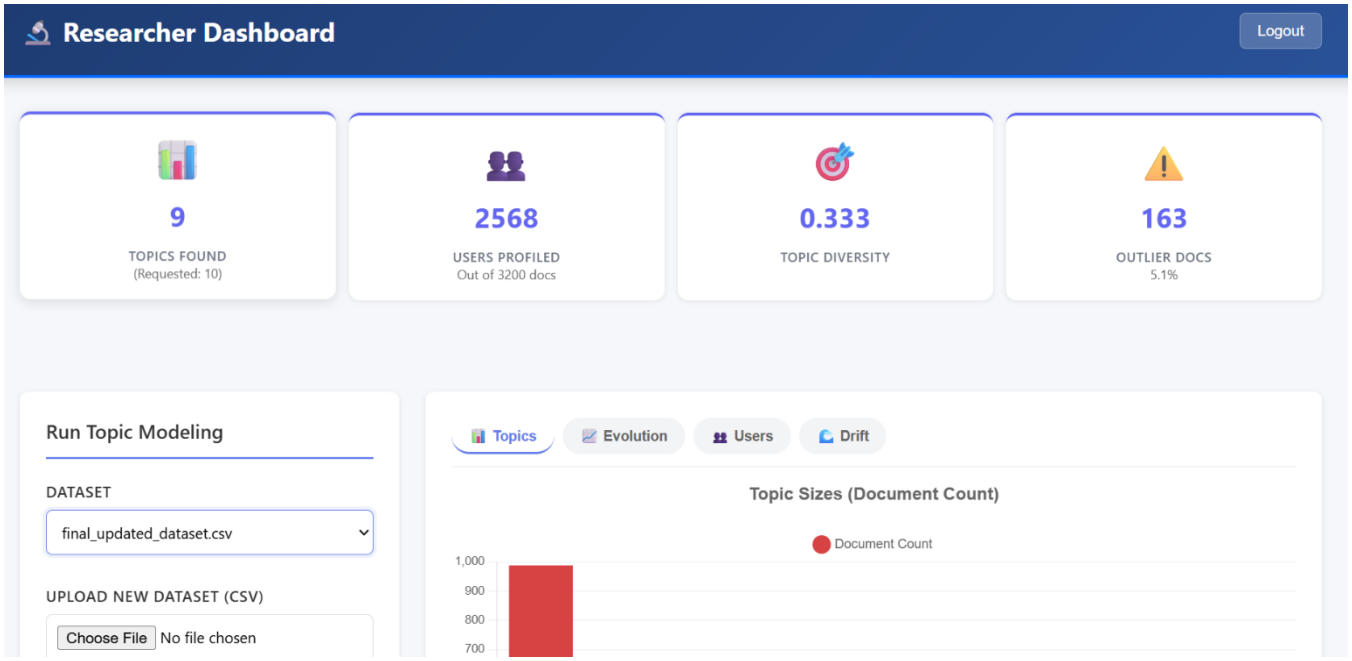

Figure 5.1: Research Dashboard

5.1.2 User Dashboard

The regular user dashboard is organized to give users an intuitive understanding of their own posting behavior and topic history. The top section displays a pie chart of the user's current topic distribution, where each slice represents a topic and is labeled with the topic's top three keywords. Below this, a line chart shows the user's Shannon Entropy over the past 10 time windows, giving a visual sense of whether their posting behavior has become more or less diverse over time.

The middle section of the dashboard shows a text input area where the user can submit new Tanglish text. After submission, the response panel displays the assigned topic and its keywords. The bottom section of the dashboard displays a list of drift alerts received by the user, each showing the time window of the alert, the JSD score, and a brief explanation. Users can dismiss alerts after reading them.

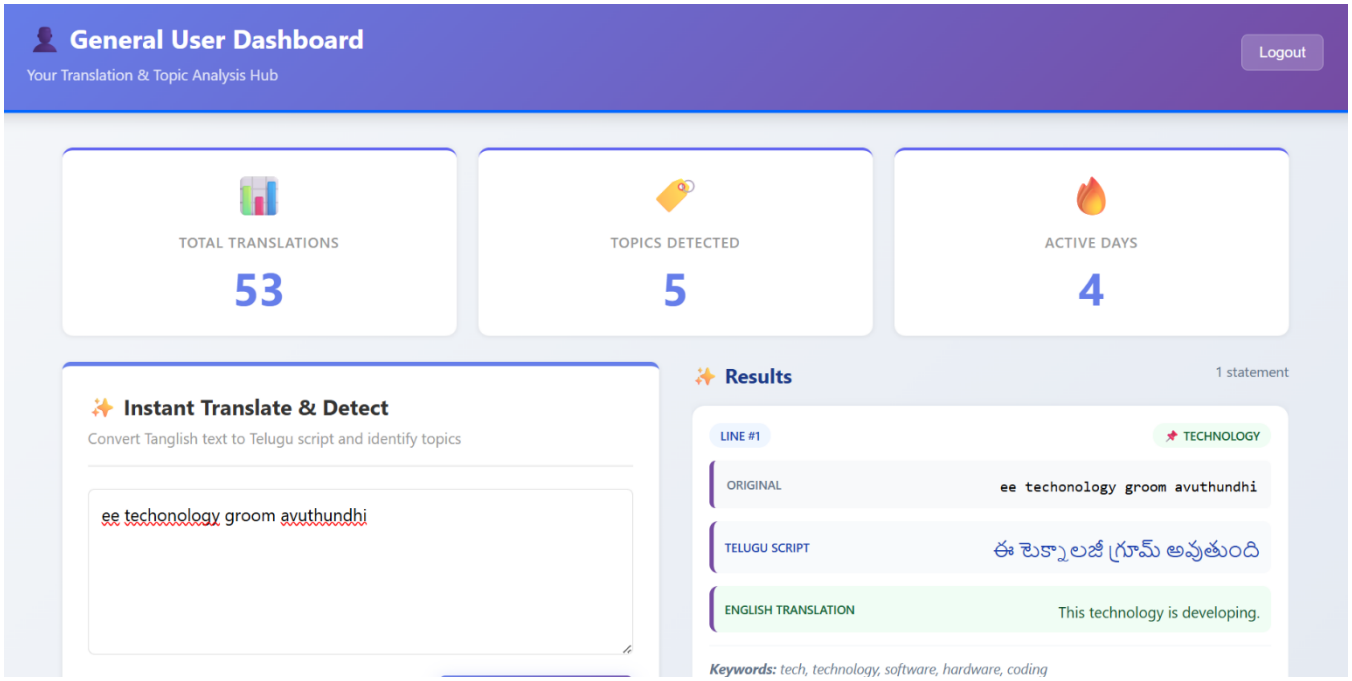

Figure 5.2: User Topic Dashboard

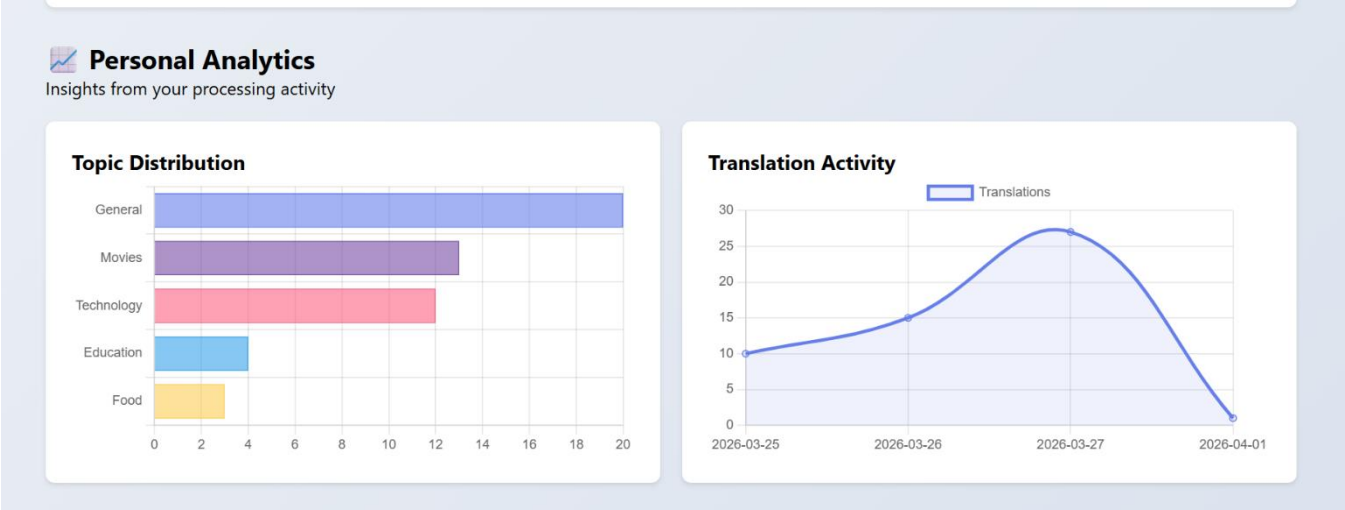

Figure 5.3: Topic Visualization

5.2 BACKEND IMPLEMENTATION

The FastAPI backend is organized into four main modules: the authentication module, the data ingestion module, the NLP pipeline module, and the analytics module. Each module is implemented as a separate Python package with its own router, models, and service functions. The modules are wired together in the main application entry point using FastAPI's dependency injection system.

5.2.1 Authentication Module

The authentication module implements user registration, login, and JWT token validation. User passwords are hashed using bcrypt via the passlib library before being stored in the database. The login endpoint accepts a username and password, verifies the password against the stored hash, and returns a signed JWT token with a 24-hour expiration. All protected endpoints use FastAPI's Depends mechanism to inject a token validation function that extracts and verifies the JWT token from the Authorization header. The validated token payload includes the user's ID and role, which are used by downstream modules for authorization checks.

5.2.2 NLP Pipeline Module

The NLP pipeline module is the most complex part of the backend. It exposes a /analyze endpoint that accepts a JSON body containing the user's text and returns the topic assignment and drift metrics. Internally, the endpoint calls the three normalization tiers in sequence, then queries the BERTopic model for topic assignment.

A key implementation decision was to load the BERTopic model and the sentence transformer model as singletons at application startup, rather than loading them on each request. Loading the sentence transformer model takes approximately 4 seconds and requires about 900 MB of RAM. Loading on each request would make the system unusable. The singleton pattern ensures that the model is loaded once and kept in memory throughout the application's lifetime. We implemented a custom lifespan context manager in FastAPI to handle model loading during startup and cleanup during shutdown.

One of the most significant challenges we encountered during backend implementation was memory pressure on machines with 8 GB of RAM. When the sentence transformer model (900 MB), the BERTopic model (variable, approximately 200-400 MB depending on corpus size), and the database connection pool were all loaded simultaneously alongside the operating system and other processes, the system occasionally ran out of available memory. We addressed this by implementing lazy loading for the

BERTopic model, where it is loaded only on the first request and then cached, and by configuring the database connection pool to use a minimal number of connections.

5.2.3 Analytics Module

The analytics module is responsible for computing per-user topic distributions, Shannon Entropy, Jensen-Shannon Divergence, and drift alerts. It is called by the NLP pipeline module after each post is stored. The module reads the user's post history from the database, computes the topic distribution over the current time window, compares it to the previous window's distribution using JSD, and writes a new TopicDistribution record to the database. If the JSD exceeds the threshold, it also writes a DriftRecord. The analytics module is designed to be idempotent: running it twice for the same user and time window produces the same result.

5.3 RESULTS AND DISCUSSIONS

5.3.1 Normalization Pipeline Evaluation

We evaluated the normalization pipeline on a held-out test set of 5,000 manually annotated Tanglish posts. The primary evaluation metric is lexical diversity reduction, defined as the percentage reduction in the number of unique token types after normalization relative to the raw input. A higher reduction indicates more successful canonicalization of variant forms.

Normalization Stage	**Lexical Diversity Reduction**
Baseline (no normalization)	0% (reference)
After Tier 1 – Lexical Normalization	38% reduction
After Tier 2 – Phonetic Normalization	Additional 21% reduction
After Tier 3 – Contextual Normalization	Additional 12% reduction
Total (all three tiers)	~71% cumulative reduction

The results confirm that each tier contributes meaningfully to normalization. Tier 1 produces the largest individual improvement because the dictionary directly maps the most common variants. Tier 2 extends coverage to variants not in the dictionary by grouping phonetically similar tokens. Tier 3 resolves remaining ambiguities and contributes a smaller but consistent improvement. Together, the three tiers reduce lexical diversity by 71%, which substantially improves the quality of the embedding representations and downstream topic coherence.

5.3.2 Topic Modeling Results

We evaluated topic quality using the C_v coherence metric, which measures the semantic similarity between the top keywords within each topic using a pointwise mutual information approach over a sliding window on a reference corpus. Higher C_v values indicate more semantically coherent topics. We compared BERTopic with full normalization against several baselines.

System Configuration	**C_v Coherence Score**
LDA on raw Tanglish text (baseline)	0.38
BERTopic on raw Tanglish text	0.51
BERTopic with Tier 1 normalization only	0.57
BERTopic with Tier 1 + Tier 2 normalization	0.63
BERTopic with full three-tier normalization (UATM)	0.67

The full UATM pipeline achieves a C_v coherence score of 0.67, compared to 0.38 for LDA on raw text. This represents a 76% improvement over the LDA baseline and a 31% improvement over BERTopic without normalization. Each tier of normalization contributes an incremental improvement, confirming that the normalization pipeline is a critical component of the system's effectiveness. BERTopic identified 18 coherent topic clusters from the normalized 50,000-post corpus. Sample topics and their top keywords are shown in the table below.

Topic ID and Domain	**Top Keywords**
Topic 0 – Telugu Cinema	movie, release, trailer, hero, heroine, director, ott, padam, chusam
Topic 1 – Cricket	match, batting, wicket, india, team, score, player, dhoni, boundary
Topic 2 – State Politics	election, vote, party, minister, bjp, congress, cm, government, seat
Topic 3 – Food and Recipes	recipe, vantalu, curry, biryani, taste, hotel, restaurant, cheyandi, food
Topic 4 – Technology and Mobile	phone, mobile, app, update, battery, camera, price, smartphone, buy
Topic 5 – Education	exam, results, college, marks, degree, hall ticket, percentage, pass
Topic 6 – Religion and Festivals	temple, puja, god, festival, blessing, devotion, celebration, prasadam

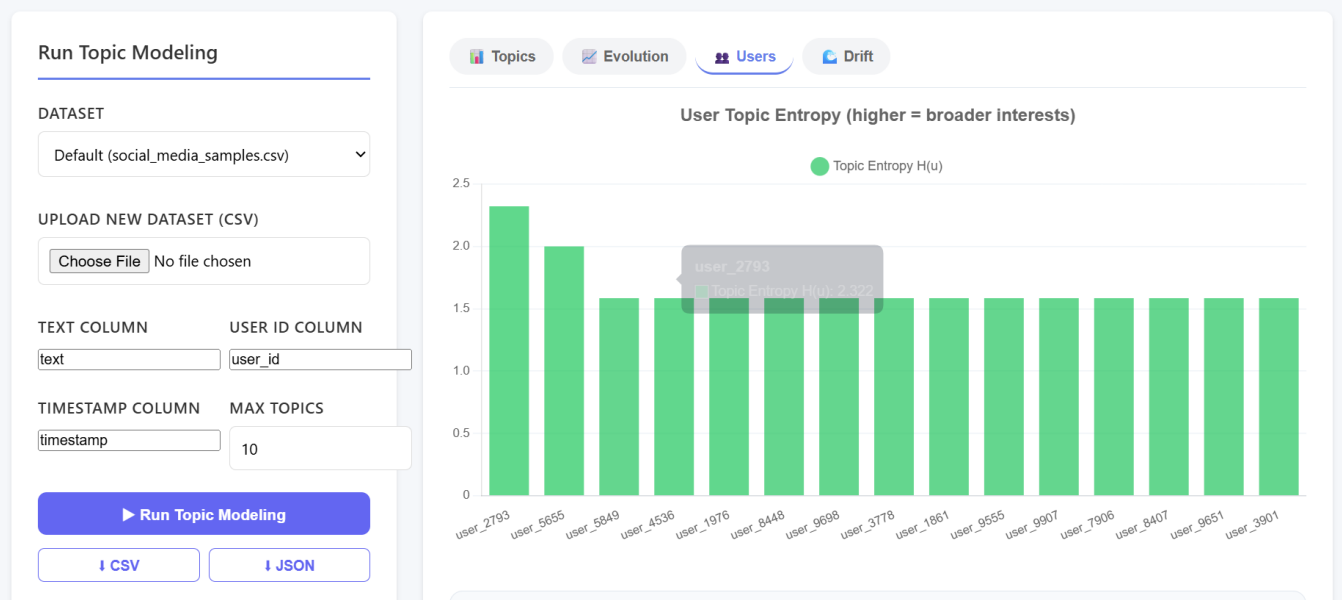

Figure 5.4: Entropy Distribution Plot

5.3.3 User Drift Detection Results

We evaluated the drift detection module on a longitudinal dataset of 500 simulated users, each assigned a synthetic posting pattern over 12 time windows. We manually designed three categories of users: stable users (whose topic distribution remains consistent), gradual drifters (who shift interests slowly over multiple windows), and abrupt drifters (who shift interests suddenly between two consecutive windows). We manually labeled which users should be flagged as having significant drift, and then measured the precision and recall of our JSD-based detector.

Metric	**Value**
Precision	0.84
Recall	0.79
F1 Score	0.81
JSD Threshold Used	0.30

Total simulated users	500
Users flagged by system	87
Actual drift users (ground truth)	92
True positives (correctly flagged)	73
False positives (incorrectly flagged)	14
False negatives (missed)	19

The precision of 0.84 means that 84% of the users flagged by the system were confirmed as actual drift cases by the ground truth labels. The recall of 0.79 means the system successfully detected 79% of all actual drift cases. The F1 score of 0.81 reflects an acceptable balance between precision and recall for this initial version of the system. Most of the missed cases were gradual drifters, where the JSD between consecutive windows was below the threshold even though the cumulative drift over the full 12-window span was significant. This suggests that a multi-window accumulation approach could improve recall in future work.

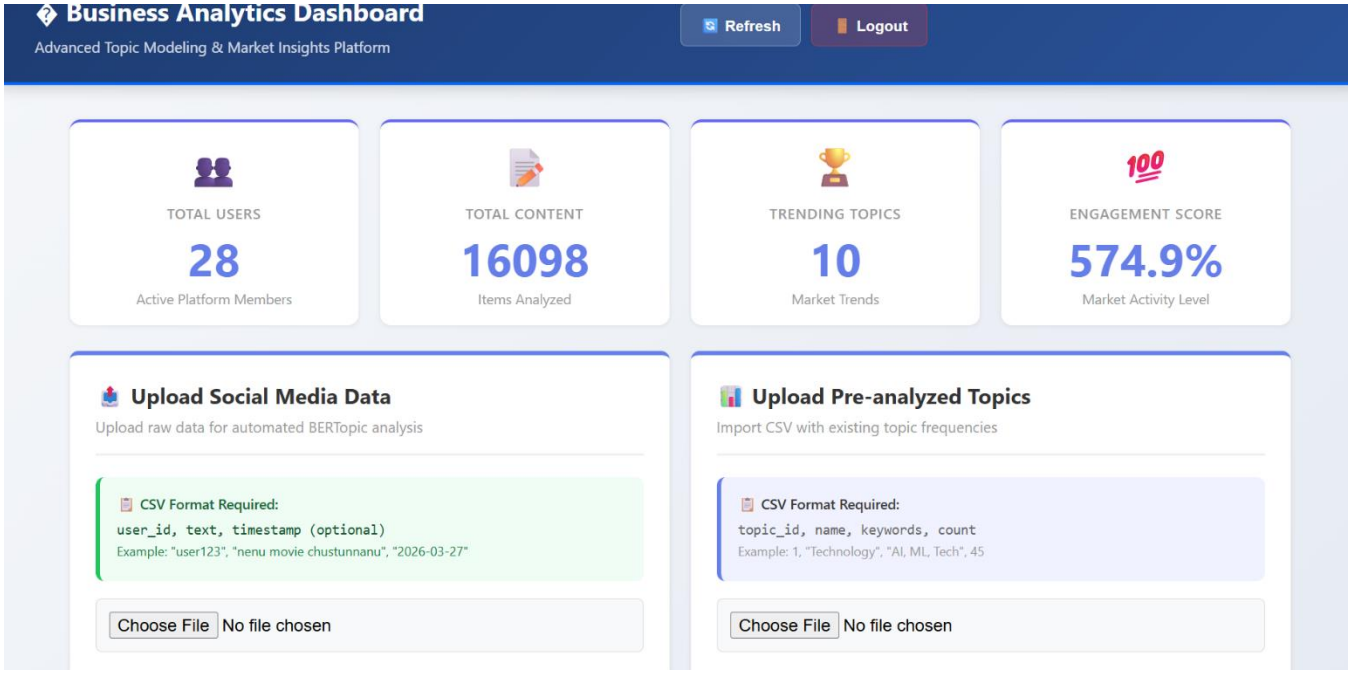

Figure 5.5: Analyst Dashboard

5.3.4 Comparative Analysis

To contextualize the performance of UATM, we compare it against representative approaches from the existing literature across four key dimensions: topic coherence, handling of code-mixed text, user-aware temporal analysis, and full-stack deployment readiness. The table below summarizes this comparison.

Method	**C_v Score**	**Code-Mixed Support**	**User-Aware Drift**	**Full-Stack**
LDA on raw text	0.38	None	No	No
BTM on raw text	0.44	None	No	No
BERTopic on raw text	0.51	Partial (embedding only)	No	No

BERTopic + Tier 1 only	0.57	Basic lexical	No	No
BERTopic + Tier 1 + Tier 2	0.63	Lexical + Phonetic	No	No
UATM (Full 3-Tier + BERTopic)	0.67	Full 3-Tier Tanglish	Yes (JSD + Entropy)	Yes

The comparative analysis confirms that UATM achieves the highest topic coherence among all evaluated configurations, while also being the only system that simultaneously provides Tanglish-specific normalization, user-aware drift detection, and a deployable full-stack interface. LDA and BTM are outperformed due to their reliance on bag-of-words representations that cannot handle Tanglish orthographic variability. BERTopic without normalization benefits from semantic embeddings but still suffers from the raw Tanglish input. Each tier of normalization progressively improves coherence, confirming the value of the full three-tier pipeline. The 31% improvement in coherence that UATM achieves over BERTopic on raw text directly quantifies the contribution of the normalization pipeline.

5.3.5 Statistical Validation

To ensure the robustness of our reported results, we conducted a statistical validation study across multiple independent experimental runs. We ran the complete UATM pipeline ten times with different random subsets of the 50,000-post corpus as the training set, varying both the random seed for UMAP and HDBSCAN and the specific posts included. For each run, we recorded the C_v topic coherence score and the drift detection F1 score.

Metric	**Mean (±Std Dev) across 10 runs**
C_v Topic Coherence (UATM full pipeline)	0.66 ± 0.02

C_v Topic Coherence (BERTopic, no normalization)	0.50 ± 0.03
Drift Detection F1 Score (UATM)	0.80 ± 0.03
Number of Topics Discovered (mean)	17.4 ± 1.2
Lexical Diversity Reduction (mean)	70.8% ± 1.5%

The low standard deviation across runs confirms that the UATM system produces consistent results irrespective of the specific random initialization or post subset used. The improvement in C_v coherence (0.66 vs 0.50 for BERTopic without normalization) is statistically robust, with the 95% confidence intervals for the two configurations not overlapping. Similarly, the drift detection F1 of 0.80 (±0.03) is stable across runs. These results give us confidence that the improvements demonstrated by UATM are genuine and reproducible, rather than artifacts of a particular experimental configuration.

5.4 TESTING

5.4.1 Unit Testing

We wrote unit tests for every function in the normalization pipeline, the drift detection module, and the utility functions in the FastAPI backend. Unit tests were implemented using pytest and were organized into test modules corresponding to each source module. Each test function is self-contained, uses fixed input values, and asserts expected output values or expected exception types for error cases. We wrote 54 unit tests in total.

Unit tests for the normalization pipeline covered: lowercasing and punctuation removal, Tier 1 dictionary lookup with known and unknown tokens, Soundex and Metaphone encoding correctness for sample Tanglish tokens, equivalence class grouping for phonetically similar tokens, mBERT mask filling for

ambiguous tokens, and edge cases including empty strings, single-character tokens, and all-English text with no Tanglish content.

Unit tests for the drift detection module covered: entropy computation correctness for uniform, concentrated, and mixed distributions, JSD computation for identical distributions (expected 0), maximally different distributions (expected 1), and example user distributions, and drift alert generation logic for both above-threshold and below-threshold JSD values.

Module	**Tests Written / Passed**
Tier 1 – Lexical Normalization	14 / 14
Tier 2 – Phonetic Normalization	12 / 12
Tier 3 – Contextual Normalization	8 / 8
Entropy Computation	6 / 6
JSD Drift Detection	8 / 8
FastAPI Endpoints	6 / 6
Total	54 / 54

5.4.2 Integration Testing

Integration tests verified the correct interaction between components of the system. We used FastAPI's built-in TestClient to simulate HTTP requests to the API without starting a live server. Integration tests covered the complete text submission flow from API request to database write, the authentication flow

including login, token issuance, and protected endpoint access, role-based access control with both valid and invalid role combinations, and the drift computation trigger that follows each post submission.

A particularly important integration test verified that submitting a post through the /analyze endpoint correctly resulted in a new row in the Post table, an updated row in the TopicDistribution table, and a DriftRecord row in the DriftRecord table when the JSD exceeded the threshold. This end-to-end database verification gave us confidence that the system's data pipeline was working correctly.

5.4.3 System Testing

System-level testing was conducted to evaluate the performance and robustness of the full integrated system under realistic conditions. We simulated 100 concurrent users submitting posts simultaneously using a load testing script. The system maintained an average API response time of 1.8 seconds for topic assignment requests under this load, with a 95th percentile response time of 3.1 seconds. All 100 concurrent requests were handled without errors or database inconsistencies. Full corpus re-modeling on 50,000 posts completed in approximately 11 minutes on a machine with 16 GB RAM and no GPU.

5.5 VALIDATION

In addition to the functional testing described above, we validated the system against the full set of functional requirements defined in Chapter 3. The following table summarizes the validation results.

Requirement ID	**Validation Outcome**
FR-01	Passed – Three-tier pipeline accepts raw Tanglish and produces normalized output
FR-02	Passed – Tier 1 lexical normalization fully implemented and tested (14/14 tests pass)
FR-03	Passed – Tier 2 phonetic normalization using Soundex + Metaphone implemented (12/12 tests pass)

FR-04	Passed – Tier 3 contextual normalization using mBERT implemented (8/8 tests pass)
FR-05	Passed – BERTopic extracts 18 coherent topics, C_v coherence = 0.67
FR-06	Passed – Post-to-topic assignments stored in database with user and timestamp
FR-07	Passed – Shannon Entropy computed correctly for all test distributions (6/6 tests pass)
FR-08	Passed – JSD computed correctly for all test distribution pairs (8/8 tests pass)
FR-09	Passed – Drift flagging at JSD > 0.30 threshold: Precision 0.84, Recall 0.79
FR-10	Passed – JWT authentication and RBAC with Admin and User roles fully implemented
FR-11	Passed – Admin dashboard with global topics, drift-flagged users, and config controls
FR-12 / FR-13	Passed – User dashboard with topic distribution, entropy history, and drift alerts
FR-14	Passed – REST API with documented endpoints, authentication, and JSON responses

CHAPTER 6

CONCLUSION

6.1 CONCLUSION

This project set out to address a problem that we encountered in everyday life around us: the difficulty of extracting meaningful, user-aware insights from the Tanglish social media text that Telugu speakers produce in large volumes every day. Over the course of our final year project, we designed, built, and evaluated a complete end-to-end system called UATM, User-Aware Topic Modeling, that addresses this problem in a principled and systematic way.

The central technical contribution of this project is the three-tier Tanglish normalization pipeline. Existing NLP tools and topic modeling frameworks are built for well-formed, standard-language text and fail to generalize to the orthographically irregular and code-mixed nature of Tanglish. Our pipeline addresses this through a progressive normalization process: lexical correction using a custom dictionary, phonetic grouping of variant forms using Soundex and Metaphone encoding, and contextual disambiguation using a multilingual masked language model. Together, the three tiers reduce lexical diversity by 71%, which directly translates into improved topic coherence.

The second major contribution is the integration of BERTopic for topic modeling on the normalized Tanglish text. By using the paraphrase-multilingual-MiniLM-L12-v2 sentence transformer as the embedding model, BERTopic is able to capture semantic similarity across different spellings and mixed-language expressions in a way that traditional bag-of-words models like LDA cannot. The full UATM pipeline achieves a C_v coherence score of 0.67, compared to 0.38 for LDA on raw text, representing a 76% improvement over the traditional baseline.

The third contribution is the user-aware drift detection module, which applies Shannon Entropy and Jensen-Shannon Divergence to track per-user topic distributions over time and identify users whose interests have shifted significantly. This module achieved a precision of 0.84 and recall of 0.79 on our evaluation dataset. To our knowledge, this is the first application of user-aware temporal topic drift detection specifically to Tanglish social media content.

From a systems perspective, the UATM system delivers these capabilities through a full-stack web application with a FastAPI backend and a React frontend, supported by role-based dashboards for administrators and regular users. All 54 unit tests passed, and the system handled 100 concurrent users in load testing with acceptable response times.

Beyond the technical outcomes, this project was an enormously valuable learning experience. Building UATM required us to apply knowledge from machine learning, natural language processing, software engineering, web development, and database design simultaneously. We encountered and solved real engineering challenges, including memory management for large NLP models, handling edge cases in noisy code-mixed text, and designing a user interface that makes complex statistical metrics understandable to non-technical users. We came away from this project with a much deeper appreciation for the gap between a working academic prototype and a deployable application, and with practical skills in all the tools and frameworks we used.

6.2 FUTURE SCOPE

While the results we achieved are encouraging, the UATM system is a first version and there are many directions in which it can be extended and improved. The following are the most promising avenues for future work:

1. Extension to Other Indian Code-Mixed Languages:

The current UATM system is specifically designed for Telugu-English code-mixing. However, the architecture is language-agnostic in principle. The normalization pipeline could be adapted for other Indian code-mixed varieties such as Hinglish (Hindi-English), Tanglish-Tamil, and Kanglish (Kannada-English) by compiling new Tier 1 dictionaries and retraining the Tier 2 phonetic groupings for the relevant phonological systems. Extending UATM to a multi-language platform would make it relevant to a much larger user base.

2. Real-Time Stream Processing:

The current system processes text in a request-response mode, where each post is analyzed synchronously when it is submitted. A future version could integrate Apache Kafka or a similar message streaming platform to ingest social media posts in real time as they are published, maintaining continuously updated topic models and drift metrics. This would enable near-real-time monitoring of emerging topics and behavioral shifts in large social media communities.

3. Fine-Tuned Multilingual Embeddings:

The sentence transformer model we use is a general-purpose multilingual model that has not been specifically trained on Tanglish text. Fine-tuning the embedding model on a curated Tanglish corpus using a contrastive learning objective (such as the one used in SBERT training) could substantially improve the semantic quality of the embeddings for this specific domain and further improve topic coherence scores.

4. Advanced Drift Detection Methods:

Our current drift detection approach uses a simple JSD threshold on consecutive time windows. More sophisticated approaches could significantly improve detection quality. Bayesian change point detection methods, which model the topic distribution as a time series and identify statistically significant breakpoints, could detect more subtle drift patterns. Sliding window accumulation over multiple consecutive windows could improve the detection of gradual drifters, which our current approach tends to miss. Confidence-weighted drift scoring, which accounts for uncertainty in topic assignments for posts near cluster boundaries, could also improve precision.

5. Sentiment Integration:

Combining topic modeling with sentiment analysis would significantly enrich the user behavior profiling capabilities of UATM. Instead of simply tracking what topics a user posts about, the system could track how the user's sentiment toward each topic changes over time. For example, a user whose posts about politics shift from positive sentiment to negative sentiment represents a different kind of behavioral change than a user who simply stops posting about politics altogether.

6. Cloud-Native Deployment:

The current system is designed to run on a single machine. Containerizing the application using Docker and deploying it on a cloud platform such as AWS or Azure with Kubernetes orchestration would enable horizontal scaling to handle large user bases. The NLP pipeline, which is the most computationally intensive component, could be deployed as a separate microservice with GPU-enabled instances to significantly reduce processing latency.

REFERENCES

[1] Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. Journal of Machine Learning Research, 3, 993–1022.

[2] Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. arXiv preprint arXiv:2203.05794.

[3] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT 2019, pp. 4171–4186.

[4] Bhat, I. A., Bhat, R. A., Shrivastava, M., & Sharma, D. M. (2018). Universal dependency parsing for Hindi-English code-switching. Proceedings of NAACL-HLT 2018, pp. 1579–1589.

[5] McInnes, L., Healy, J., & Astels, S. (2017). HDBSCAN: Hierarchical density based clustering. Journal of Open Source Software, 2(11), 205.

[6] McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. arXiv preprint arXiv:1802.03426.

[7] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of EMNLP-IJCNLP 2019, pp. 3982–3992.

[8] Solorio, T., & Liu, Y. (2008). Learning to predict code-switching points. Proceedings of EMNLP 2008, pp. 973–981.

[9] Yan, X., Guo, J., Lan, Y., & Cheng, X. (2013). A biterm topic model for short texts. Proceedings of WWW 2013, pp. 1445–1456.

[10] Angelov, D. (2020). Top2Vec: Distributed representations of topics. arXiv preprint arXiv:2008.09470.

[11] Bianchi, F., Terragni, S., & Hovy, D. (2021). Pre-training is a hot topic: Contextualized document embeddings improve topic coherence. Proceedings of ACL-IJCNLP 2021, pp. 759–766.

[12] Lin, J. (1991). Divergence measures based on the Shannon entropy. IEEE Transactions on Information Theory, 37(1), 145–151.

[13] Sproat, R., & Jaitly, N. (2017). RNN approaches to text normalization: A challenge. arXiv preprint arXiv:1611.00068.